

ABSTRACT

The project, referred to as Automated and Partition Aware ETL Pipeline for IPL data, presents the design and implementation of an automated data pipeline for processing IPL data. It integrates open-source technologies including Apache Hadoop, Apache Spark (PySpark), Apache Airflow and PostgreSQL. The goal of the system is to automate data ingestion, transformation and storage in a structured manner to support analytical tasks.

It is assumed that Hadoop (HDFS) is the landing server where the file comes and saved. Then the pipeline begins by scanning for new data files in HDFS. Once files are detected, PySpark scripts are triggered through Airflow to perform transformation such as cleaning, enrichment and partitioning of data. The transformed data is then loaded into PostgreSQL for further querying and analysis.

The system is designed in such a way that when a csv file reaches to HDFS folder only then the PySpark script is triggered otherwise Airflow will skip the tasks related to that csv file.

To ensure reliability, the system incorporates error handling, success and failure emails notification and logging mechanisms.

Additionally, a reporting feature summarizes the number of records processed and inserted during each run.

This project demonstrates the practical application of data engineering concept such as distribution storage, scalable processing, workflow orchestration and real-time integration. The outcome is a strong and reusable pipeline that can be extended for other sports datasets or similar use case where large-scale, structured data processing is required.

CONTENTS

	Page No.
1. Chapter 01: Introduction	11
<i>Problem Statement</i>	
<i>Objectives</i>	
<i>Scope of the Project</i>	
<i>Significance of the Study</i>	
2. Chapter 02: Background and Related Work	13
<i>Distributed Storage with Hadoop</i>	
<i>Data Transformation with PySpark</i>	
<i>Structured Storage with PostgreSQL</i>	
<i>Workflow Orchestration with Apache Airflow</i>	
<i>Related Work in Sports Analytics</i>	
3. Chapter 03: Installation and Environment Setup	14
<i>Oracle Virtual Machine</i>	
<i>CentOS 9 Environment</i>	
<i>Apache Hadoop Installation</i>	
<i>PySpark Installation</i>	
<i>Apache Airflow Installation</i>	
<i>PostgreSQL Installation</i>	
<i>pgAdmin Installation</i>	
<i>PowerBI Installation</i>	
<i>Integrated Architecture Overview</i>	
<i>Chapter Summary</i>	
4. Chapter 04: Raw Data Collection and Structure Design	22
<i>Data Collection</i>	
<i>Entity-Relationship (ER) Diagram</i>	
<i>Chapter Summary</i>	
5. Chapter 05: Database and Table Creation	26
<i>Database Setup</i>	
<i>Table Creation</i>	
<i>Chapter Summary</i>	
6. Chapter 06: Hadoop (HDFS)	31
<i>Benefits of Hadoop</i>	
<i>Hadoop as the Landing Server</i>	
<i>Folder Structure and Partitioning</i>	
<i>Chapter Summary</i>	
7. Chapter 07: PySpark Scripts	33
<i>Why Spark is Used</i>	
<i>PySpark vs Pandas</i>	
<i>PySpark Scripts</i>	
<i>Chapter Summary</i>	
8. Chapter 08: Partitioning and Python Script	38
<i>Purpose of Partitioning</i>	
<i>Linking Partitions with Database Tables</i>	
<i>The partition_deletion.py script</i>	
<i>Working Principle of script</i>	
<i>Key Features</i>	
<i>Example Scenario</i>	
<i>Chapter Summary</i>	

9. Chapter 09: Apache Airflow and Email Alerts	42
<i>Benefits of Airflow</i>	
<i>DAGs Explanation</i>	
<i>Mailing Alerts</i>	
<i>Chapter Summary</i>	
10. Chapter 10: PowerBI	45
<i>Benefits of PowerBI</i>	
<i>Use Case in my Project IPL_db</i>	
<i>Chapter Summary</i>	
11. Chapter 11: End Results	46
<i>Key Outputs of the Project</i>	
<i>Chapter Summary</i>	
12. Chapter 12: Discussion	53
<i>Advantages of the Project</i>	
<i>Why using multiple tools is better than a single ETL tool</i>	
<i>Comparison with typical ETL Projects</i>	
<i>Chapter Summary</i>	
13. Chapter 13: Summary	56
14. Chapter 14: Conclusion	58
15. Reference	59
16. Check List Item for Final Report	60

Chapter 1

Introduction

The rapid growth of data in the digital age has transformed the way organizations collect, process, and analyse information. In domains such as e-commerce, finance, etc., efficient data pipelines are critical for transforming raw and large-scale datasets into insights. Sports analytics has also emerged as a significant area where data-driven methods are increasingly used for performance evaluation, strategic decision-making, and fan engagement. Cricket generates vast amounts of structured and semi-structured data in the form of match records, ball-by-ball commentary, and player statistics. Handling this data requires strong data engineering solutions capable of ensuring scalability, automation, and reliability.

The Indian Premier League (IPL) represents one of the richest sources of cricket data, with detailed records of multiple seasons. Traditional methods of analysing cricket datasets often rely on manual querying and standalone scripts. While useful for small-scale analysis, these methods are limited in scalability, error handling, and reproducibility. Consequently, there is a need for automated pipelines that can efficiently manage ingestion, transformation, and storage of such data.

This dissertation focuses on the design and implementation of an **automated and partition-aware ETL (Extract, Transform, Load) pipeline** for IPL datasets. The pipeline integrates four major technologies:

- **Hadoop** for distributed storage of raw and processed data and for partitioning
- **PySpark** for large-scale transformation and cleaning.
- **PostgreSQL** for structured relational storage.
- **Apache Airflow** for scheduling and monitoring of pipeline.

The project aims to create a system that not only automates repetitive data processing tasks but also ensures transparency through partitioning and logging. By storing outputs in timestamped partitions within HDFS and recording run metadata in PostgreSQL, the pipeline maintains reproducibility and auditability.

1.1 Problem Statement

Sports datasets such as IPL are large, dynamic, and contain inconsistencies that require intensive cleaning and transformation. Existing approaches lack automation and struggle with scaling to millions of records. This project addresses the problem by building an automated ETL pipeline that integrates distributed processing, structured storage, and workflow orchestration.

1.2 Objectives

The main objectives of this dissertation are:

1. To set up an integrated environment combining Hadoop, PySpark, PostgreSQL, and Airflow.
2. To design database tables for IPL data.
3. To develop PySpark and Python scripts for cleaning, transforming, and loading data into PostgreSQL.

4. To implement HDFS partitioning and logging mechanisms to ensure transparency and to track historical data.
5. To orchestrate the entire pipeline using Airflow for automation and monitoring.
6. To include check file system so that the tasks only triggered when a file reached to its folder.

1.3 Scope of the Project

The scope of this work is limited to batch oriented ETL processes for IPL datasets stored in CSV format. While the system is tested in a local VM and Windows hybrid setup, they can be migrated to cloud environments. Real-time streaming, visualization, and predictive analytics are beyond the immediate scope but are potential areas for future work.

1.4 Significance of the Study

The significance of this project lies in integrating modern data engineering practices with the domain of sports analytics. By applying distributed storage, parallel processing, and workflow automation to IPL data, the project demonstrates how scalable pipelines can unlock new opportunities for researchers, analysts, and fans. The resulting framework can be adapted for other large-scale datasets in cricket or extended to different sports.

Chapter 2

Background and Related Work

The development of automated data pipelines has its roots in the broader field of data warehousing and business intelligence. Early approaches to ETL (Extract, Transform, Load) were based on manual scripts and batch jobs, which were rigid and often difficult to maintain. With the rise of big data, new frameworks and distributed architectures became essential to meet the challenges of scale, speed, and fault tolerance.

2.1 Distributed Storage with Hadoop

Hadoop emerged as a cornerstone technology for big data, offering a distributed file system (HDFS) and parallel processing through MapReduce. HDFS ensures data replication, fault tolerance, and scalability, making it a reliable backbone for large-scale analytics. In the context of IPL datasets, Hadoop provides a scalable foundation for storing historical and future cricket data efficiently. [1]

2.2 Data Transformation with PySpark

PySpark, the Python API for Spark, extends this capability to data scientists and engineers comfortable with Python. Its DataFrame API allows developers to apply complex transformations, manage schema evolution, and integrate easily with other storage systems such as HDFS and relational databases. For IPL data, PySpark enables operations such as cleaning inconsistent records, aggregating match statistics, and computing advanced performance metrics. [2]

2.3 Structured Storage with PostgreSQL

While distributed systems handle raw and semi-structured data efficiently, relational databases remain central for structured analytical queries. PostgreSQL is a widely used open-source RDBMS. Its features, including primary and foreign key constraints, indexing, and advanced query optimization, ensure that stored data maintains integrity and is easily accessible. For this project, PostgreSQL serves as the target database, where cleaned IPL data is stored that can later be used for visualization and statistical analysis. [3]

2.4 Workflow Orchestration with Apache Airflow

Apache Airflow introduced the concept of workflows as Directed Acyclic Graphs (DAGs), making it easier to define task dependencies, schedule jobs, and monitor execution. Airflow also provides extensibility through operators and sensors that integrate with multiple systems. In the proposed project, Airflow ensures that each stage—file ingestion, PySpark transformation, HDFS partitioning, and loading into PostgreSQL—runs in the correct sequence with proper logging and error handling. [4]

2.5 Related Work in Sports Analytics

Most prior work in sports analytics, particularly in cricket, has relied on traditional programming and SQL queries for data processing. While effective for small datasets, these approaches often lack scalability and automation. For instance, several studies focused on player performance modelling or match outcome prediction, but these pipelines were often manually executed. By contrast, this project applies a complete big data stack (Hadoop, PySpark, PostgreSQL, and Airflow) to build an automated, partition-aware ETL pipeline.

Chapter 3

Installation And Environment Setup

This chapter presents a comprehensive overview of the installation and configuration process of all the software tools and environments used in this project. The purpose of this setup is to create a stable, scalable, and integrated environment for data engineering and workflow orchestration tasks. The tools include *Oracle Virtual Machine*, *CentOS 9*, *Apache Hadoop*, *Apache PySpark*, *Apache Airflow*, *PostgreSQL*, *pgAdmin* and *PowerBI*.

Each tool was carefully selected for its stability, compatibility, and community support. The combination of these tools provides an end-to-end solution capable of handling the Extract, Transform, Load (ETL) process in an automated and fault-tolerant manner. Whereas PowerBI is used to create dashboard to visualize data that is stored in the database.

The following sections describe each component, its purpose, technical specifications, download sources, and its role within the integrated environment.

3.1 Oracle Virtual Machine

Oracle VM VirtualBox provides the foundation for hosting the CentOS 9 environment. It enables virtualization of hardware resources such as CPU, memory, and storage, thereby isolating the experimental setup from the host operating system.

The use of a virtual machine ensures reproducibility and system stability, particularly important for research-based projects. VirtualBox allows for resource allocation, network configuration, and snapshot creation, making it ideal for development and testing environments. [5]

- **Official Website:** <https://www.virtualbox.org/>
- **Version Used:** Oracle VM VirtualBox 7.1
- **Host Operating System:** Windows 11 (64-bit)
- **Guest Operating System:** CentOS Stream 9 (64-bit)
- **Allocated Resources:** 10 GB RAM, 2 CPU cores, 50 GB storage

This configuration provides a controlled and secure sandbox where all subsequent installations and configurations can be performed without affecting the host system.

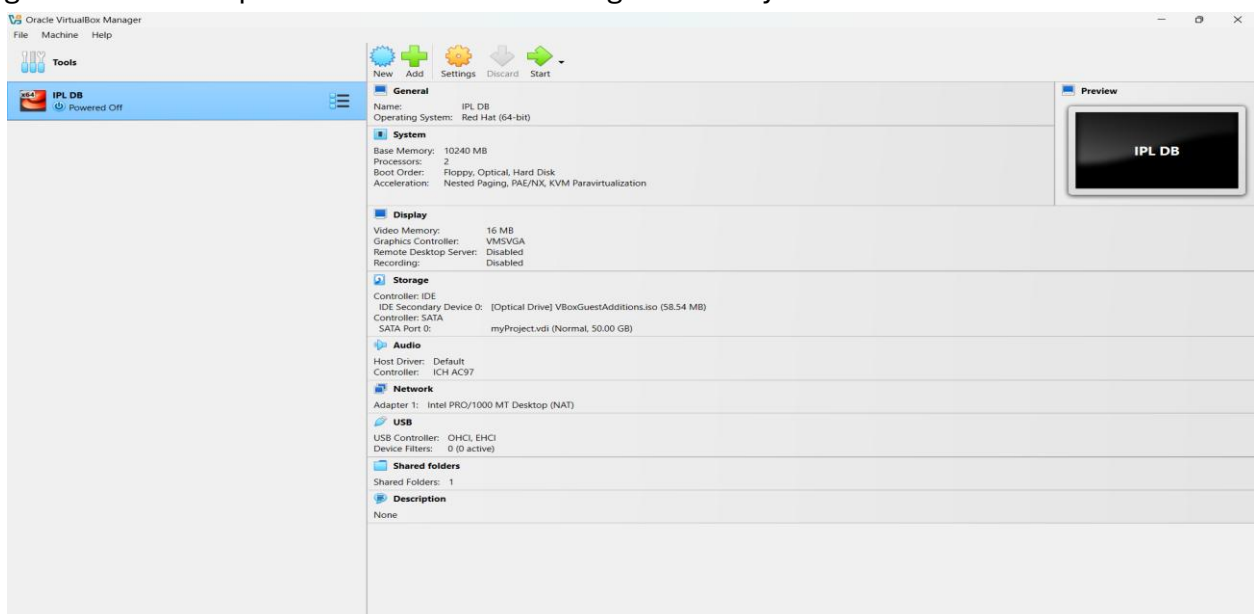


Fig 3.1. Oracle VM Virtual Box homepage

3.2 CentOS 9 Environment

CentOS Stream 9, an open-source Linux distribution, serves as the operating platform for the entire setup. It provides the system libraries, network tools, and package management utilities required by all installed software.

CentOS 9 was selected due to its binary compatibility with Red Hat Enterprise Linux (RHEL), making it highly stable and enterprise friendly. It offers long-term support, efficient memory handling, and strong compatibility with Hadoop, Spark, and PostgreSQL. [6]

Key system characteristics:

- **Official Download Link:** <https://www.centos.org/download/>
- **Version:** CentOS Stream 9
- **Kernel:** Linux 5.9
- **Architecture:** x86_64 (64-bit)
- **Package Manager:** DNF
- **Desktop Environment:** Minimal installation (CLI-based)

Post-installation configuration included network setup, enabling SSH services, and creation of system users such as *hdfs*, *pyspark* and *airflow* for tool-specific operations.

3.3 Apache Hadoop Installation

3.3.1 Overview

Apache Hadoop is the core component of the data processing layer, providing distributed file storage (HDFS) and computation (MapReduce). It enables scalable data processing across multiple nodes and offers reliability through data replication.

In this environment, Hadoop was configured in a pseudo-distributed mode, allowing all services (NameNode, DataNode, ResourceManager, and NodeManager) to run on a single virtual machine. This mode provides a realistic simulation of a cluster environment while remaining suitable for development purposes. [1]

3.3.2 Technical Details

- **Version Used:** Hadoop 3.3.6
- **Programming Language:** Java
- **Required Dependency:** OpenJDK 17
- **Installation Directory:** /home/hadoop
- **Core Components:**
 - **HDFS:** Hadoop Distributed File System for storage
 - **YARN:** Yet Another Resource Negotiator for resource management
 - **MapReduce:** Data computation model

3.3.3 Configuration Overview

- **File System Default:** hdfs://localhost:9000
- **Replication Factor:** 1 (for single-node mode)

Hadoop's environment variables (e.g., HADOOP_HOME, JAVA_HOME, and PATH) were added to the system's shell configuration to ensure global accessibility.

3.3.4 Download Information

- **Official Website:** <https://hadoop.apache.org/releases.html>
- **Binary Distribution:** `hadoop-3.3.6.tar.gz`

```
[hadoop@vbox ~]$ hdfs dfs
Usage: hadoop fs [generic options]
[-appendToFile [-n] <localsrc> ... <dst>]
[-cat [-ignoreCrc] <src> ...]
[-checksum [-v] <src> ...]
[-chgrp [-R] GROUP PATH...]
[-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
[-chown [-R] [OWNER][:[GROUP]] PATH...]
[-concat <target path> <src path> <src path> ...]
[-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>] <localsrc> ... <dst>]
[-copyToLocal [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>] [-q <thread pool queue size>] <src> ... <localdst>]
[-count [-q] [-h] [-v] [-t <storage type>]] [-u] [-x] [-e] [-s] <path> ...]
[-cp [-f] [-p | -p[topax]] [-d] [-t <thread count>] [-q <thread pool queue size>] <src> ... <dst>]
[-createSnapshot <snapshotDir> [<snapshotName>]]
[-deleteSnapshot <snapshotDir> <snapshotName>]
[-df [-h] [<path> ...]]
[-du [-s] [-h] [-v] [-x] <path> ...]
[-expunge [-immediate] [-fs <path>]]
[-find <path> ... <expression> ...]
[-get [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>] [-q <thread pool queue size>] <src> ... <localdst>]
[-getfacl [-R] <path>]
[-getfattr [-R] {-n name | -d} [-e en] <path>]
[-getmerge [-nl] [-skip-empty-file] <src> <localdst>]
[-head <file>]
[-help [cmd ...]]
[-ls [-C] [-d] [-h] [-q] [-R] [-t] [-S] [-r] [-u] [-e] [<path> ...]]
[-mkdir [-p] <path> ...]
[-moveFromLocal [-f] [-p] [-l] [-d] <localsrc> ... <dst>]
[-moveToLocal <src> <localdst>]
[-mv <src> ... <dst>]
[-put [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>] <localsrc> ... <dst>]
[-renameSnapshot <snapshotDir> <oldName> <newName>]
[-rm [-f] [-r|-R] [-skipTrash] [-safely] <src> ...]
[-rmdir [--ignore-fail-on-non-empty] <dir> ...]
[-setfacl [-R] [{-b|-k} {-m|-x <acl_spec>} <path>][--set <acl_spec> <path>]]
[-setfattr {-n name [-v value] | -x name} <path>]
[-setrep [-R] [-w] <rep> <path> ...]
[-stat [format] <path> ...]
[-tail [-f] [-s <sleep interval>] <file>]
[-test [-defswrz] <path>]
[-text [-ignoreCrc] <src> ...]
[-touch [-a] [-m] [-t TIMESTAMP (yyyyMMdd:HHmmss) ] [-c] <path> ...]
[-touchz <path> ...]
[-truncate [-w] <length> <path> ...]
[-usage [cmd ...]]

Generic options supported are:
-conf <configuration file>      specify an application configuration file
-D <property=value>             define a value for a given property
-fs <file:///|hdfs://namenode:port> specify default filesystem URL to use, overrides 'fs.defaultFS' property from configurations.
-jt <local|resourcemanager:port> specify a ResourceManager
-files <file1,...>               specify a comma-separated list of files to be copied to the map reduce cluster
-libjars <jar1,...>              specify a comma-separated list of jar files to be included in the classpath
-archives <archive1,...>        specify a comma-separated list of archives to be unarchived on the compute machines
```

Fig 3.2. HDFS Installation

3.4 PySpark Installation

3.4.1 Overview

PySpark is the Python API for Apache Spark, providing an efficient and high-level interface for distributed data processing. It was selected for this project due to its seamless integration with Hadoop and Python's extensive data manipulation capabilities.

PySpark operates as the computational engine responsible for transforming and aggregating data stored in HDFS. It supports in-memory computation, which significantly speeds up processing compared to traditional MapReduce. [2]

3.4.2 Technical Details

- **Version Used:** Spark 4.0.0
- **Programming Language:** Scala, Java, and Python

- **Execution Mode:** Local (master = local[*])
- **Cluster Manager:** Standalone (can be integrated with YARN)
- **Installation Directory:** /usr/local/spark
- **Environment Variables:**
 - SPARK_HOME=/usr/local/spark
 - PYSPARK_PYTHON=python3

PySpark interacts directly with HDFS for reading input files and writing output data. It was also configured to write transformed datasets into PostgreSQL tables using JDBC connectors.

```
Python 3.9.23 (main, Aug 19 2025, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-11)] on linux
Type "help", "copyright", "credits" or "license()" for more information.
WARNING: Using incubator modules: jdk.incubator.vector
25/10/18 00:46:30 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to

      ____
     /___/\_--_____/___\
    _\ \/_\_ \_\_\' ___/'__\
   /__ / .__/\_,_/_/_/_\ \ version 4.0.0
    /_/_

Using Python version 3.9.23 (main, Aug 19 2025 00:00:00)
Spark context Web UI available at http://vbox:4040
Spark context available as 'sc' (master = local[*], app id = local-1760728591583).
SparkSession available as 'spark'.
>>>
```

Fig 3.3. PySpark

3.4.3 Download Information

- **Official Website:** <https://spark.apache.org/downloads.html>
- **Selected Package:** Pre-built for Hadoop 3.x
- **Download Example:** spark-3.5.1-bin-hadoop3.tgz
- **License:** Apache License 2.0

This configuration ensures compatibility between PySpark and Hadoop, allowing both systems to operate cohesively within the CentOS environment.

3.5 Apache Airflow Installation

3.5.1 Overview

Apache Airflow is the orchestration tool used to automate, schedule, and monitor the project's data workflows. It enables the creation of Directed Acyclic Graphs (DAGs), where each node represents a task in the data pipeline.

Airflow was chosen for its extensibility, modularity, and integration capabilities with Hadoop, Spark, and PostgreSQL. It supports task dependencies, retries, and notifications, ensuring reliability and traceability in workflow execution. [4]

3.5.2 Technical Details

- **Version Used:** Apache Airflow 2.9.x
- **Python Compatibility:** Python 3.9
- **Metadata Database:** PostgreSQL (production-ready)
- **Scheduler and Webserver Ports:**
 - Webserver: <http://localhost:8080>

- **Environment Variables:**

- AIRFLOW_HOME=~/.airflow

3.5.3 Download Information

- **Official Website:** <https://airflow.apache.org/>

- **Distribution:** Python package (via PyPI: apache-airflow)

```

-----
----  |__ ( ) -----  __/___/____
----  /| |  /___  __/___/___  /___  \_ | /| / /
----  ___ |  /___/  _ __/___/ / / / / | / | /
___/___/ |___/___/ /___/___/ \____/____/|___/

Running the Gunicorn Server with:
Workers: 4 sync
Host: 0.0.0.0:8080
Timeout: 120
Logfiles: - -
Access Logformat:
=====

```

Fig 3.4. Airflow

3.6 PostgreSQL Installation

3.6.1 Overview

PostgreSQL is the relational database management system (RDBMS) used in this project to store structured and processed data. It was selected due to its strong compliance with SQL standards, support for ACID transactions, and performance with analytical workloads.

Within this environment, PostgreSQL acts as the destination layer in the ETL pipeline. Processed datasets from PySpark are loaded into PostgreSQL tables through JDBC connections, enabling further analysis and reporting. [3]

3.6.2 Technical Details

- **Version Used:** PostgreSQL 17
- **Default Port:** 5432
- **Storage Engine:** MVCC (Multi-Version Concurrency Control)
- **Database Encoding:** UTF-8
- **Authentication:** Role-based
- **Integration:** Accessible to Airflow and PySpark

3.6.3 Download Information

- **Official Website:** <https://www.postgresql.org/download/>
- **Linux Repository:** Available through DNF and YUM package managers
- **License:** PostgreSQL License (open-source)

PostgreSQL ensures reliable data persistence and query performance, forming the backbone of the structured data storage in the project.

3.7 pgAdmin Installation

3.7.1 Overview

pgAdmin is the graphical user interface for PostgreSQL database management. It provides an intuitive and user-friendly platform for database design, querying, monitoring, and maintenance. In this project, pgAdmin was used for visualizing database structures, verifying data loads from PySpark jobs, and validating the results of Airflow workflows. Its interactive dashboard supports SQL execution, performance monitoring, and data export. [7]

3.7.2 Technical Details

- **Version Used:** pgAdmin 4
- **Interface Type:** Web-based GUI
- **Default Port:** 5050
- **Authentication:** Email-based login (configured locally)
- **Integration:** Connects to PostgreSQL using host localhost, port 5432, and user credentials defined earlier.

3.7.3 Download Information

- **Official Website:** <https://www.pgadmin.org/download/>
- **License:** PostgreSQL License

pgAdmin simplifies the management of the PostgreSQL environment and allows visual validation of all data pipelines integrated with Airflow and PySpark.

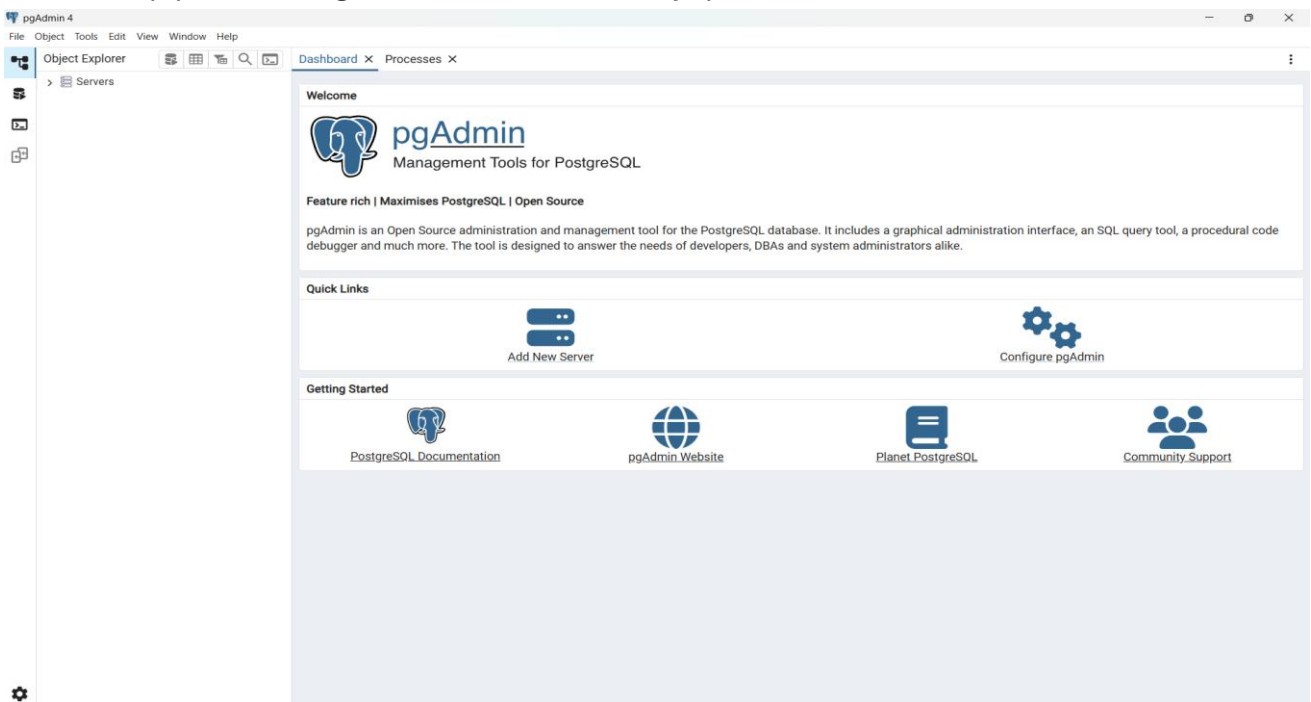


Fig 3.5. pgAdmin4 homepage

3.8 Power BI Installation

Microsoft Power BI is a business intelligence and data visualization tool used to transform raw data into interactive reports and dashboards. It integrates easily with PostgreSQL through an on-premises data gateway, allowing real-time access to processed data for analytics and insights.

In this setup, Power BI is used to connect to the PostgreSQL database, visualize trends, and monitor ETL pipeline results. The reports refresh automatically via scheduled updates, reflecting the most recent data ingested through Airflow and PySpark. [8]

- **Download Link:** <https://powerbi.microsoft.com/desktop/>
- **Purpose:**
 - To visualize PostgreSQL data through dynamic dashboards.
 - To enable real-time reporting using the on-premises data gateway.
 - To provide insights into ETL outcomes and overall data quality.

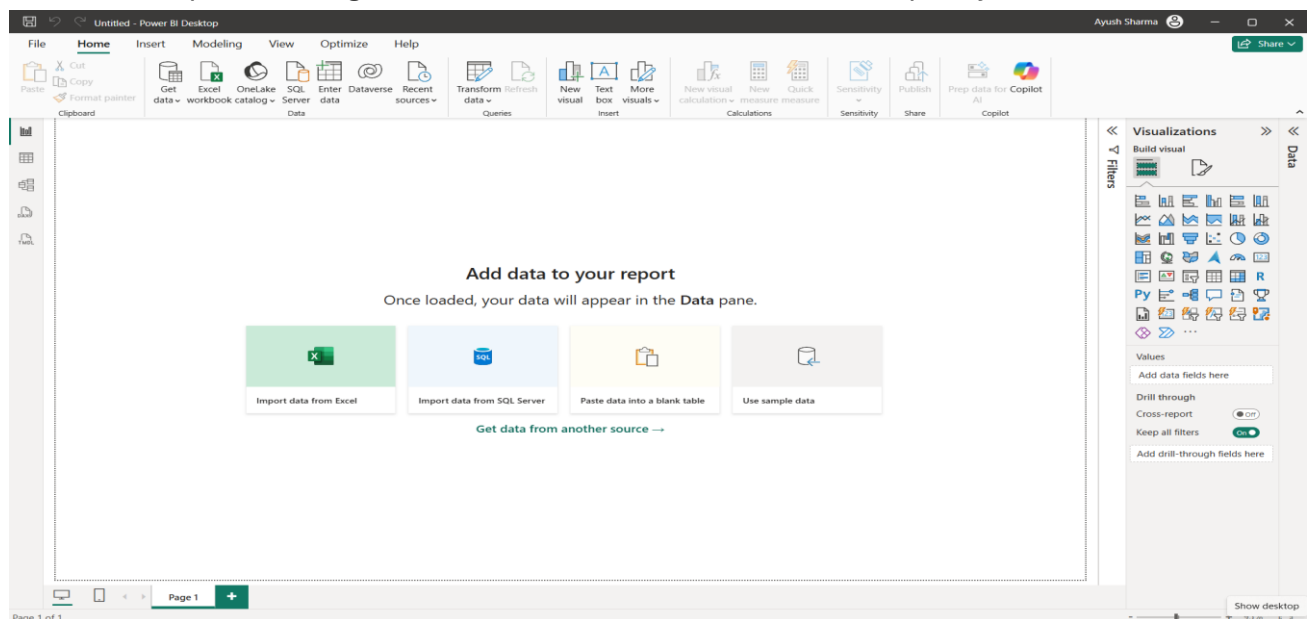


Fig 3.6. PowerBI homepage

3.9 Integrated Architecture Overview

The complete environment operates as a layered architecture designed to ensure smooth data flow and automation across all components.

Layer	Tool	Function
Infrastructure	Oracle VM VirtualBox	Provides isolated virtual environment
Operating System	CentOS 9	Hosts all core services
Data Storage	Hadoop (HDFS)	Distributed file system for raw data
Processing Engine	PySpark	Performs transformations and analytics
Orchestration	Apache Airflow	Automates and schedules data workflows
Database	PostgreSQL	Stores processed and structured data
Database GUI	pgAdmin	Provides user interface for database management

Table 3.1. Environment Overview

Data flows through the system in the following sequence:

1. **Input Data** is placed in Hadoop's distributed file system (HDFS).

2. **PySpark** reads the data, performs transformations, and loads the results into **PostgreSQL**.
3. **Airflow** orchestrates this workflow by executing DAGs at scheduled intervals.
4. **pgAdmin** is used to validate the data loaded in PostgreSQL and visualize schema-level changes.

This integrated setup closely resembles real-world data engineering environments and demonstrates the interoperability between big data and workflow automation tools.

3.10 Chapter Summary

This chapter described the environment used for the project, focusing on the technical configuration and integration of each component.

The tools, Hadoop, PySpark, Airflow, PostgreSQL, pgAdmin and PowerBI were selected to provide a strong, scalable, and fully automated data engineering ecosystem.

Together, they enable distributed processing, reliable scheduling, and efficient data management. The integration across these technologies establishes the foundation for the project's implementation phase, ensuring a seamless flow of data from ingestion to storage and visualization.

Chapter 4

Raw Data Collection and Structure Design

This chapter presents the process of collecting raw data and designing the structured data model for the project. The goal was to transform unstructured and semi-structured data collected from multiple sources into a well-organized relational schema that supports efficient querying, analysis, and visualization.

The raw data used in this study primarily pertains to **Indian Premier League (IPL)** matches, batsmen, bowlers and ball-by-ball deliveries. The datasets were sourced from **Kaggle** and the **official IPL website**, both of which provide publicly available and detailed historical records of the tournament. This chapter explains how the raw data was collected, pre-processed, and structured into relational tables. It also includes an Entity-Relationship (ER) diagram representing the logical relationships between key entities in the system.

4.1 Data Collection

The raw data was obtained from two primary sources:

1. **Kaggle IPL Dataset:**

Kaggle hosts several IPL-related datasets that include match-level and delivery-level information, such as players' performances, match outcomes, and ball-by-ball details. These datasets were downloaded in CSV format. [9]

- **Source:** <https://www.kaggle.com>
- **Datasets Used:**
 - matches.csv – Match metadata such as venue, teams, toss results, and winners.
 - deliveries.csv – Ball-by-ball details including batsmen, bowlers, and runs scored.

2. **IPL Official Website:**

The **official IPL website ()** was used to verify player statistics, match outcomes, and to cross-check the data accuracy. Additional player details such as strike rates, averages, and total innings were extracted and merged with the Kaggle dataset. [10]

- **Source:** <https://www.iplt20.com>
- **Datasets Used:**
 - batsmen.csv – batsmen metadata which includes batsmen name and batsmen's stats.
 - bowlers.csv – bowlers metadata which includes bowlers name and bowler's stats.

All datasets were subjected to validation and transformation steps before integration into the PostgreSQL database.


```

batsmen_id,batsmen_name,total_runs,total_matches,total_innings,total_not_outs,average,total_balls_faced,strike_rate,hundreds,fifties,fours,sixes
1,Aakash Chopra,53,7,6,0,9.7,71,69.33,0,0,7,0
2,Aaron Finch,2091,92,90,7,22.63,1631,119.69,0,15,214,78
3,AB de Villiers,4697,169,157,36,37.08,3048,147.29,2,37,374,239
4,Abdul Basith,1,1,1,1,0,1,100,0,0,0,0
5,Abdul Samad,577,50,40,10,19.4,395,131.2,0,0,37,36
6,Abdur Razzak,0,1,1,1,0,2,0,0,0,0,0
7,Abhijeet Tomar,4,1,1,0,4,8,50,0,0,1,0
8,Abhimanyu Mithun,34,14,8,3,5.33,26,120,0,0,4,1
9,Abhinav Manohar,231,19,15,1,13.83,174,112.53,0,0,21,10
10,Abhinav Mukund,19,3,2,0,9.5,22,45.24,0,0,1,0
11,Abhishek Jhunjhunwala,217,21,16,3,12.44,210,86.03,0,1,19,5
12,Abhishek Nayar,672,60,50,12,17.89,577,107.82,0,0,55,20
13,Abhishek Raut,194,22,16,7,22.5,167,115.99,0,0,13,7
14,Abhishek Sharma,1377,63,61,7,25.9,887,147.09,0,7,128,73
15,Abhishek Porel,360,18,16,2,20.48,236,132.98,0,2,37,15
16,Abu Nechim,14,7,4,3,5,6,300,0,0,3,0
17,Adam Gilchrist,2069,80,80,4,28.13,1495,137.35,2,11,239,92
18,Adam Milne,23,8,6,2,5.75,29,76.22,0,0,0,1
19,Adam Voges,181,9,7,3,45.25,143,126.57,0,0,15,3
20,Adam Zampa,15,17,5,0,2.78,24,44.64,0,0,1,0
21,Adil Rashid,22,2,2,0,11,16,137.5,0,0,2,1
22,Aditya Dole,34,3,2,0,17,22,154.54,0,0,3,1
23,Aditya Tare,339,35,28,4,11.64,273,104.64,0,1,40,11
24,Adrian Barath,42,3,3,1,21,42,100,0,0,5,1
25,Aiden Blizard,120,6,6,0,20,87,137.93,0,1,21,2
26,Aiden Markram,995,44,42,9,30.96,767,127.98,0,5,73,35
27,Ajantha Mendis,3,9,3,1,0.67,6,50,0,0,0,0
28,Ajinkya Rahane,4642,185,171,17,26.4,3761,115.79,2,30,478,103
29,Ajit Agarkar,179,38,21,11,20.56,154,112.55,0,0,13,5
30,Ajit Chandila,4,8,1,1,0.66,66,0,0,0,0
31,Akash Deep,19,3,2,1,17,10,156.25,0,0,2,1
32,Akash Madhwal,4,5,1,1,1,5,80,0,0,0,0

```

Fig 4.1. batsmen.csv sample data

```

bowler_id,bowler_name,total_wkts,total_matches,total_inns,total_overs,runs,bowling_avg,economy_rate,strike_rate,total_4w,total_5w
1,Aaron Finch ,1,14,3,4,49,49,12.25,24,0,0
2,Aavishkar Salvi ,3,2,2,8,66,22,8.25,16,0,0
3,Abdul Samad ,1,12,3,7,96,96,13.71,42,0,0
4,Abhimanyu thun ,7,14,14,41,397,46.73,10.56,27.6,0,0
5,Abhishek Jhunjhunwala ,1,5,3,6,38,38,6.33,36,0,0
6,Abhishek Nayar ,9,48,18,37,306,41.5,8.36,28.63,0,0
7,Abhishek Sharma ,11,46,23,41,351,34.8,8.72,22.8,0,0
8,Abu Nechim ,7,9,9,33,286,60.22,8.32,38.22,0,0
9,Adam lne ,7,9,9,32,308,43.77,9.89,26.67,0,0
10,Adam Zampa ,29,20,20,70,558,25.59,7.97,18.75,0,1
11,Adil Rashid ,2,2,2,7,56,28,8,21,0,0
12,Aditya Dole ,5,3,3,11,112,22.4,10.18,13.2,0,0
13,Aiden Markram ,2,27,10,14.1,134,67,9.62,42.5,0,0
14,Ajantha Mendis ,8,10,10,40,285,33.75,7.02,29,0,0
15,Ajit Agarkar ,29,38,38,120,1059,39.5,8.85,26.48,0,0
16,Ajit Chandila ,11,12,12,39,242,21.6,6.19,20.9,1,0
17,Akash Deep ,7,8,8,26.8,319,51.67,12.8,24.53,0,0
18,Akash Madhwal ,19,13,13,43.4,424,28.32,9.93,16.36,1,1
19,Akash Singh ,5,6,6,19,188,37.6,9.89,22.8,0,0
20,Akeal Hosein ,1,1,1,4,40,40,10,24,0,0
21,Akshath Reddy ,1,5,1,3,32,32,10.66,18,0,0
22,Albie Morkel ,85,91,87,286,2355,29.31,8.66,20.65,1,0
23,Alfonso Thomas ,14,15,15,52,406,31.67,7.75,24.5,0,0
24,Ali Murtaza ,9,11,11,42,302,33.55,7.61,28,0,0
25,Alzai Joseph ,21,22,22,71.4,691,49.84,10.02,28.24,0,1
26,Anand Rajan ,8,8,8,25,201,24.66,8.15,19.22,0,0
27,Andre Nel ,1,1,1,3,31,31,10.33,18,0,0
28,Andre Russell ,115,124,109,274.4,2578,37.42,9.65,22.95,2,1
29,Andrew Flintoff ,2,3,3,11,105,52.5,9.54,33,0,0
30,Andrew McDonald ,11,9,8,26,216,18.93,8.15,13.6,0,0
31,Andrew Symonds ,20,35,27,81,573,31.62,7.97,23.69,0,0
32,Andrew Tye ,42,30,30,114,979,42.31,9.5,25.1,3,1
33,Angelo Mathews ,27,45,41,124,992,37.45,7.92,28.33,1,0
34,Aniket Choudhary ,5,5,5,17,144,28.8,8.55,20.2,0,0
35,Anil Kumble ,45,42,42,160,1058,27.96,6.74,24.03,2,1
36,Ankeet Chavan ,8,11,11,36,299,48.36,7.82,38.71,0,0
37,Ankit Rajpoot ,24,29,29,88,814,44.14,9.17,27.38,0,1
38,Ankit Sharma .12.19.18.57.406.43.5.6.99.35.56.0.0

```

Fig 4.2. bowlers.csv sample data

match_id,city,date,match_type,player_of_match,batsmen_id,bowlers_id,venue,team1,team2,toss_winner,toss_decision,winner,result_margin,target_runs,target_overs,super_over,method,umpire1,umpire2

335982,Bangalore,18-04-2008,Match 1,BB McCullum,115,NULM,Chinnaswamy Stadium,Royal Challengers Bangalore,Kolkata Knight Riders,Royal Challengers Bangalore,field,Kolkata Knight Riders,runs,140,223,20,N,NA,Asad Rauf,RE Koertzen

335983,Chandigarh,19-04-2008,Match 2,MEK Hussey,351,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Chennai Super Kings,Chennai Super Kings,bat,Chennai Super Kings,runs,33,241,20,N,NA,MR Benson,SL Shastri

335984,Delhi,19-04-2008,Match 3,MH Aharof,348,NULM,Feroz Shah Kotla,Delhi Daredevils,Rajasthan Royals,Rajasthan Royals,bat,Delhi Daredevils,wickets,9,138,20,N,NA,Aleem Dar,GA Pratapkumar

335985,Mumbai,20-04-2008,Match 4,JV Boucher,338,NULM,Wankhede Stadium,Mumbai Indians,Royal Challengers Bangalore,Mumbai Indians,bat,Royal Challengers Bangalore,wickets,5,166,20,N,NA,SJ Davis,DJ Harper

335986,Kolkata,20-04-2008,Match 5,DS Hussey,152,NULM,Eden Gardens,Kolkata Knight Riders,Deccan Chargers,bat,Kolkata Knight Riders,wickets,5,111,20,N,NA,BF Bowden,K Hariharan

335987,Jaipur,21-04-2008,Match 6,SN Watson,529,NULM,Sawai Mansingh Stadium,Rajasthan Royals,Kings XI Punjab,Kings XI Punjab,bat,Rajasthan Royals,wickets,6,167,20,N,NA,Aleem Dar,RB Tiffin

335988,Hyderabad,22-04-2008,Match 7,V Sehwag,637,NULM,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Delhi Daredevils,Deccan Chargers,bat,Delhi Daredevils,wickets,9,143,20,N,NA,IL Howell,AM Saheba

335989,Chennai,23-04-2008,Match 8,MJ Marsh,343,NULM,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Mumbai Indians,Mumbai Indians,field,Chennai Super Kings,runs,6,209,20,N,NA,DJ Harper,GA Pratapkumar

335990,Hyderabad,24-04-2008,Match 9,YK Pathan,658,NULM,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Rajasthan Royals,Rajasthan Royals,field,Rajasthan Royals,wickets,3,215,20,N,NA,Asad Rauf,MR Benson

335991,Chandigarh,25-04-2008,Match 10,KC Sangakkara,301,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Mumbai Indians,Mumbai Indians,field,Kings XI Punjab,wickets,6,183,20,N,NA,Aleem Dar,AM Saheba

335992,Bangalore,26-04-2008,Match 11,SR Watson,529,NULM,Chinnaswamy Stadium,Royal Challengers Bangalore,Rajasthan Royals,Rajasthan Royals,field,Rajasthan Royals,wickets,7,136,20,N,NA,MR Benson,IL Howell

335993,Chennai,26-04-2008,Match 12,JPD Oram,232,NULM,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Kolkata Knight Riders,Kolkata Knight Riders,bat,Chennai Super Kings,wickets,9,148,20,N,NA,BR Doctrove,AV Jayaprakash

335994,Mumbai,27-04-2008,Match 13,AC Gilchrist,17,NULM,Dr DY Patil Sports Academy,Mumbai Indians,Deccan Chargers,Deccan Chargers,field,Deccan Chargers,wickets,10,155,20,N,NA,Asad Rauf,SL Shastri

335995,Chandigarh,27-04-2008,Match 14,SM Katich,563,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Delhi Daredevils,Delhi Daredevils,bat,Kings XI Punjab,wickets,4,159,20,N,NA,RE Koertzen,I Shivram

335996,Bangalore,28-04-2008,Match 15,MS Dhoni,375,NULM,Chinnaswamy Stadium,Royal Challengers Bangalore,Chennai Super Kings,Chennai Super Kings,bat,Chennai Super Kings,runs,13,179,20,N,NA,BR Doctrove,RB Tiffin

335997,Kolkata,29-04-2008,Match 16,ST Jayasuriya,NULM,336,Eden Gardens,Kolkata Knight Riders,Mumbai Indians,Kolkata Knight Riders,bat,Mumbai Indians,wickets,7,138,20,N,NA,BF Bowden,AV Jayaprakash

335998,Delhi,30-04-2008,Match 17,60 McGrath,NULM,125,Feroz Shah Kotla,Delhi Daredevils,Royal Challengers Bangalore,Royal Challengers Bangalore,field,Delhi Daredevils,runs,10,192,20,N,NA,Aleem Dar,I Shivram

335999,Hyderabad,01-05-2008,Match 18,SE Marsh,533,NULM,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Kings XI Punjab,Kings XI Punjab,field,Kings XI Punjab,wickets,7,165,20,N,NA,BR Doctrove,RB Tiffin

336000,Jaipur,01-05-2008,Match 19,SA Asnodkar,587,NULM,Sawai Mansingh Stadium,Rajasthan Royals,Kolkata Knight Riders,Rajasthan Royals,bat,Rajasthan Royals,runs,45,197,20,N,NA,RE Koertzen,GA Pratapkumar

336001,Chennai,02-05-2008,Match 20,V Sehwag,637,NULM,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Delhi Daredevils,Chennai Super Kings,bat,Delhi Daredevils,wickets,8,178,20,N,NA,BF Bowden,K Hariharan

336002,Hyderabad,03-05-2008,Match 21,R Vinay Kumar,NULM,429,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Royal Challengers Bangalore,Deccan Chargers,bat,Royal Challengers Bangalore,wickets,5,166,20,N,NA,Asad Rauf,RE Koertzen

336003,Chandigarh,04-05-2008,Match 22,IK Pathan,225,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Kolkata Knight Riders,Kings XI Punjab,bat,Kings XI Punjab,wickets,9,179,20,N,NA,DJ Harper,I Shivram

336004,Mumbai,04-05-2008,Match 23,SN Pollock,NULM,359,Dr DY Patil Sports Academy,Mumbai Indians,Delhi Daredevils,Delhi Daredevils,field,Mumbai Indians,runs,20,163,20,N,NA,IL Howell,RE Koertzen

336005,Jaipur,04-05-2008,Match 24,Sohail Tanvir,NULM,380,Sawai Mansingh Stadium,Rajasthan Royals,Chennai Super Kings,Chennai Super Kings,bat,Rajasthan Royals,wickets,8,118,20,N,NA,Asad Rauf,AV Jayaprakash

336006,Bangalore,05-05-2008,Match 25,S Sreesanth,NULM,356,M Chinnaswamy Stadium,Royal Challengers Bangalore,Kings XI Punjab,Kings XI Punjab,wickets,6,127,20,N,NA,SJ Davis,BR Doctrove

336007,Chennai,06-05-2008,Match 26,KC Gilchrist,17,NULM,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Deccan Chargers,Deccan Chargers,field,Deccan Chargers,wickets,7,145,20,N,NA,MR Benson,RB Tiffin

336008,Mumbai,07-05-2008,Match 27,A Mehra,NULM,47,Dr DY Patil Sports Academy,Mumbai Indians,Rajasthan Royals,Mumbai Indians,field,Mumbai Indians,wickets,7,104,20,N,NA,DJ Harper,RE Koertzen

336009,Delhi,08-05-2008,Match 28,MS Dhoni,375,NULM,Feroz Shah Kotla,Delhi Daredevils,Chennai Super Kings,Chennai Super Kings,field,Chennai Super Kings,wickets,4,188,20,N,NA,Aleem Dar,RB Tiffin

336010,Kolkata,08-05-2008,Match 29,SC Ganguly,566,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Kolkata Knight Riders,Kolkata Knight Riders,bat,Kolkata Knight Riders,runs,5,130,16,N,NA,Asad Rauf,IL Howell

336011,Jaipur,09-05-2008,Match 30,YK Pathan,658,NULM,Sawai Mansingh Stadium,Rajasthan Royals,Deccan Chargers,Rajasthan Royals,field,Rajasthan Royals,wickets,8,141,20,N,NA,MR Benson,AM Saheba

336012,Bangalore,10-05-2008,Match 31,CRD Fernando,NULM,108,M Chinnaswamy Stadium,Royal Challengers Bangalore,Mumbai Indians,Mumbai Indians,field,Mumbai Indians,wickets,9,123,18,N,NA,BF Bowden,AV Jayaprakash

336013,Chennai,10-05-2008,Match 32,L Balaji,NULM,209,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Kings XI Punjab,Kings XI Punjab,field,Chennai Super Kings,runs,18,182,20,N,NA,AV Jayaprakash,BG Jerling

336014,Hyderabad,11-05-2008,Match 33,SC Ganguly,566,NULM,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Kolkata Knight Riders,Kolkata Knight Riders,bat,Kolkata Knight Riders,runs,23,205,20,N,NA,IL Howell,AM Saheba

336015,Jaipur,11-05-2008,Match 34,SR Watson,529,NULM,Sawai Mansingh Stadium,Rajasthan Royals,Delhi Daredevils,Rajasthan Royals,field,Rajasthan Royals,wickets,3,157,20,N,NA,SJ Davis,RE Koertzen

336016,Chandigarh,12-05-2008,Match 35,SE Marsh,533,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Royal Challengers Bangalore,Royal Challengers Bangalore,bat,Kings XI Punjab,wickets,9,144,20,N,NA,BR Doctrove,I Shivram

336017,Kolkata,13-05-2008,Match 36,Shoaib Akhtar,NULM,336,Eden Gardens,Kolkata Knight Riders,Delhi Daredevils,Kolkata Knight Riders,bat,Kolkata Knight Riders,runs,23,134,20,N,NA,Asad Rauf,IL Howell

336018,Mumbai,14-05-2008,Match 37,ST Jayasuriya,NULM,336,Wankhede Stadium,Mumbai Indians,Chennai Super Kings,Mumbai Indians,field,Mumbai Indians,wickets,9,157,20,N,NA,BR Doctrove,AM Saheba

336019,Chandigarh,15-05-2008,Match 38,SE Marsh,533,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Rajasthan Royals,Rajasthan Royals,field,Kings XI Punjab,runs,41,222,20,N,NA,SJ Davis,K Hariharan

336020,Delhi,15-05-2008,Match 39,A Mishra,45,NULM,Feroz Shah Kotla,Delhi Daredevils,Deccan Chargers,Deccan Chargers,field,Delhi Daredevils,runs,12,195,20,N,NA,BG Jerling,GA Pratapkumar

336021,Mumbai,16-05-2008,Match 40,SN Pollock,NULM,359,Wankhede Stadium,Mumbai Indians,Kolkata Knight Riders,Mumbai Indians,field,Mumbai Indians,wickets,8,68,20,N,NA,BR Doctrove,DJ Harper

336022,Delhi,17-05-2008,Match 41,DPM Jayawardene,323,NULM,Feroz Shah Kotla,Delhi Daredevils,Kings XI Punjab,Delhi Daredevils,bat,Kings XI Punjab,runs,6,89,8,N,D/L,AV Jayaprakash,RE Koertzen

336023,Jaipur,17-05-2008,Match 42,KC Smith,202,NULM,Sawai Mansingh Stadium,Rajasthan Royals,Royal Challengers Bangalore,Royal Challengers Bangalore,field,Rajasthan Royals,runs,65,198,20,N,NA,BF Bowden,SL Shastri

336024,Hyderabad,18-05-2008,Match 43,D Bravo,146,NULM,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Mumbai Indians,Deccan Chargers,field,Mumbai Indians,runs,25,179,20,N,NA,BR Doctrove,DJ Harper

336025,Kolkata,18-05-2008,Match 44,M Ntini,NULM,226,Eden Gardens,Kolkata Knight Riders,Chennai Super Kings,Kolkata Knight Riders,bat,Chennai Super Kings,runs,3,53,8,N,D/L,Asad Rauf,K Hariharan

336026,Bangalore,19-05-2008,Match 45,SP Goswami,551,NULM,Chinnaswamy Stadium,Royal Challengers Bangalore,Delhi Daredevils,Delhi Daredevils,field,Delhi Daredevils,wickets,5,155,20,N,NA,SJ Davis,GA Pratapkumar

336027,Kolkata,20-05-2008,Match 46,YK Pathan,658,NULM,Eden Gardens,Kolkata Knight Riders,Rajasthan Royals,Rajasthan Royals,field,Rajasthan Royals,wickets,6,148,20,N,NA,BG Jerling,RE Koertzen

336028,Mumbai,21-05-2008,Match 47,SE Marsh,533,NULM,Wankhede Stadium,Mumbai Indians,Kings XI Punjab,Mumbai Indians,field,Kings XI Punjab,runs,1,190,20,N,NA,BF Bowden,GA Pratapkumar

336029,Chennai,21-05-2008,Match 48,A Kumble,NULM,35,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Rajasthan Royals,Rajasthan Royals,field,Rajasthan Royals,wickets,4,127,20,N,NA,DJ Harper,I Shivram

336031,Chandigarh,23-05-2008,Match 49,SE Marsh,533,NULM,"Punjab Cricket Association Stadium, Mohali",Kings XI Punjab,Deccan Chargers,Kings XI Punjab,field,Kings XI Punjab,wickets,6,176,20,N,NA,Asad Rauf,SJ Davis

336032,Delhi,24-05-2008,Match 50,KD Karthik,172,NULM,Feroz Shah Kotla,Delhi Daredevils,Mumbai Indians,Delhi Daredevils,field,Delhi Daredevils,wickets,5,177,20,N,NA,BF Bowden,K Hariharan

336033,Chennai,24-05-2008,Match 51,JA Morkel,NULM,22,"M Chidambaram Stadium, Chepauk",Chennai Super Kings,Rajasthan Royals,Rajasthan Royals,bat,Rajasthan Royals,runs,10,212,20,N,NA,DJ Harper,SL Shastri

336034,Bangalore,03-05-2008,Match 52,P Kumar,NULM,294,M Chinnaswamy Stadium,Royal Challengers Bangalore,Deccan Chargers,Deccan Chargers,field,Royal Challengers Bangalore,runs,3,157,20,N,NA,BR Doctrove,SL Shastri

336035,Kolkata,25-05-2008,Match 53,Umar Gul,NULM,414,Eden Gardens,Kolkata Knight Riders,Kings XI Punjab,Kings XI Punjab,bat,Kolkata Knight Riders,wickets,3,175,20,N,NA,SJ Davis,I Shivram

336036,Jaipur,26-05-2008,Match 54,Shoaib Tanvir,NULM,380,Sawai Mansingh Stadium,Rajasthan Royals,Mumbai Indians,Rajasthan Royals,field,Rajasthan Royals,wickets,5,146,20,N,NA,BF Bowden,K Hariharan

336037,Hyderabad,27-05-2008,Match 55,SR Raina,584,NULM,"Rajiv Gandhi International Stadium, Uppal",Deccan Chargers,Chennai Super Kings,Deccan Chargers,bat,Chennai Super Kings,wickets,7,148,20,N,NA,BG Jerling,AM Saheba

336038,Mumbai,30-05-2008,Match 56,SR Watson,529,NULM,Wankhede Stadium,Delhi Daredevils,Rajasthan Royals,Delhi Daredevils,field,Rajasthan Royals,runs,105,193,20,N,NA,BF Bowden,RE Koertzen

336039,Mumbai,31-05-2008,Match 57,Final,I Shivram,NULM,226,Wankhede Stadium,Chennai Super Kings,Kings XI Punjab,Kings XI Punjab,bat,Chennai Super Kings,wickets,9,113,20,N,NA,Asad Rauf,DJ Harper

336040,Mumbai,01-06-2008,Match 58,Final,YK Pathan,658,NULM,Dr DY Patil Sports Academy,Chennai Super Kings,Rajasthan Royals,Rajasthan Royals,field,Rajasthan Royals,wickets,3,164,20,N,NA,BF Bowden,RE Koertzen

392181,Cape Town,18-04-2009,Match 1,SR Tendulkar,497,NULM,Newlands,Chennai Super Kings,Mumbai Indians,Chennai Super Kings,field,Mumbai Indians,runs,19,166,20,N,NA,BR Doctrove,K Hariharan

Fig 4.3. matches.csv sample data

match_id,inning,batting_team,bowling_team,over,ball,batter,bowler,non_striker,batsman_runs,extra_runs,total_runs,extras_type,is_wicket,player_dismissed,dismissal_kind,fielder

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,1,SC Ganguly,P Kumar,BB McCullum,0,1,1,legbyes,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,2,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,3,BB McCullum,P Kumar,SC Ganguly,0,1,1,wides,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,4,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,5,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,6,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,7,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,1,BB McCullum,Z Khan,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,2,BB McCullum,Z Khan,SC Ganguly,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,3,BB McCullum,Z Khan,SC Ganguly,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,4,BB McCullum,Z Khan,SC Ganguly,6,0,6,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,5,BB McCullum,Z Khan,SC Ganguly,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,6,BB McCullum,Z Khan,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,1,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,2,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,3,SC Ganguly,P Kumar,BB McCullum,0,1,1,legbyes,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,4,BB McCullum,P Kumar,SC Ganguly,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,5,BB McCullum,P Kumar,SC Ganguly,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,6,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,1,BB McCullum,AA Noffke,SC Ganguly,0,5,5,wides,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,2,BB McCullum,AA Noffke,SC Ganguly,6,0,6,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,3,BB McCullum,AA Noffke,SC Ganguly,0,1,1,legbyes,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,4,SC Ganguly,AA Noffke,BB McCullum,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,5,SC Ganguly,AA Noffke,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,6,SC Ganguly,AA Noffke,BB McCullum,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,7,BB McCullum,AA Noffke,SC Ganguly,6,0,6,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,1,SC Ganguly,P Kumar,BB McCullum,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,2,SC Ganguly,P Kumar,BB McCullum,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,3,BB McCullum,P Kumar,SC Ganguly,4,0,4,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,4,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,5,BB McCullum,P Kumar,SC Ganguly,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,6,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,1,BB McCullum,Z Khan,SC Ganguly,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,2,SC Ganguly,Z Khan,BB McCullum,0,0,0,1,SC Ganguly,caught,3H Kallis

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,3,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,4,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,5,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,6,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,1,BB McCullum,AA Noffke,RT Ponting,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,2,RT Ponting,AA Noffke,BB McCullum,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,3,BB McCullum,AA Noffke,RT Ponting,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,4,RT Ponting,AA Noffke,BB McCullum,2,0,2,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,5,RT Ponting,AA Noffke,BB McCullum,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,6,BB McCullum,AA Noffke,RT Ponting,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,7,1,BB McCullum,Z Khan,RT Ponting,0,0,0,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,7,2,BB McCullum,Z Khan,RT Ponting,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,7,3,RT Ponting,Z Khan,BB McCullum,1,0,1,0,NA,NA,NA

335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,7,4,BB McCullum,Z Khan,RT Ponting,1,0,1,0,NA,NA,NA

Fig 4.4. deliveries.csv sample data

4.2 Entity-Relationship (ER) Diagram

The ER diagram for the database schema is represented below. It depicts the logical relationships among the four entities — **Matches**, **Deliveries**, **Batsmen**, and **Bowlers**.

Explanation of Relationships:

- Each **match** (Matches table) has multiple **deliveries** (Deliveries table).
- Each delivery record belongs to one match, creating a one-to-many relationship.
- Each match links to one or more **batsmen** and **bowlers** using foreign keys.
- The Batsmen and Bowlers tables store cumulative player-level performance data independent of match occurrences.

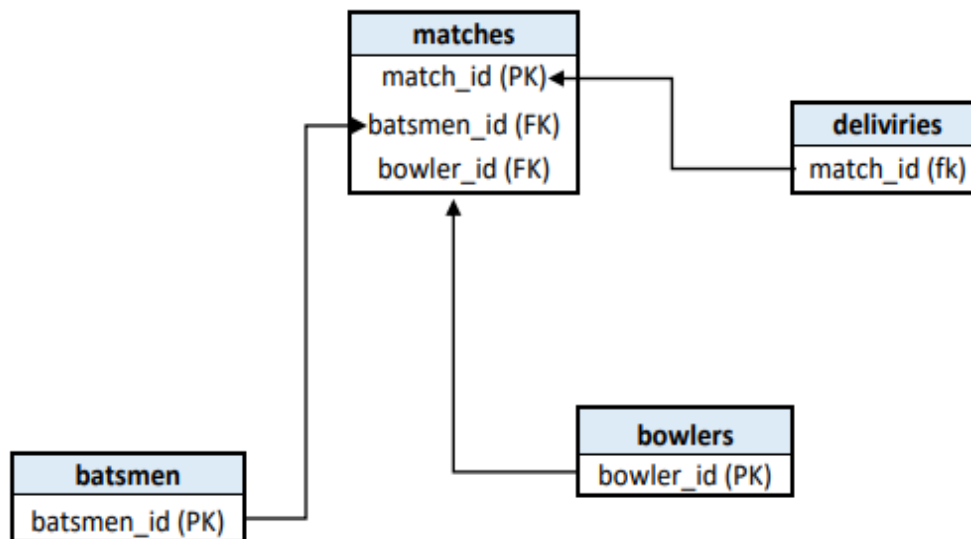


Fig 4.5. ER Diagram

4.3 Chapter Summary

This chapter detailed the raw data sources, preprocessing, and structured database design. The final schema was normalized and optimized for analytical queries, maintaining referential integrity through primary and foreign key relationships.

The resulting design supports efficient integration with big data processing tools like **PySpark** for transformation and **Airflow** for orchestration, while PostgreSQL provides reliable storage and query performance. The next chapter will describe the implementation workflow, ETL design, and data processing logic.

Chapter 5

Database and Table Creation

In this chapter, I designed and created the database structure required for the IPL data processing pipeline. PostgreSQL was used as the primary database system due to its reliability, scalability, and compatibility with PySpark and Airflow. The schema was planned to support efficient querying, auditing, and partition tracking. A total of six tables were created: **batsmen**, **bowlers**, **matches**, **deliveries**, **hdfs_partition_log**, and **script_execution_log**.

5.1 Database Setup

The PostgreSQL database was initialized with a dedicated schema named IPL_db. Each table was created with proper data types, primary keys, and foreign key relationships to maintain referential integrity. This setup ensures smooth data ingestion from PySpark scripts and consistent integration with Airflow DAGs for automated ETL execution. [7]

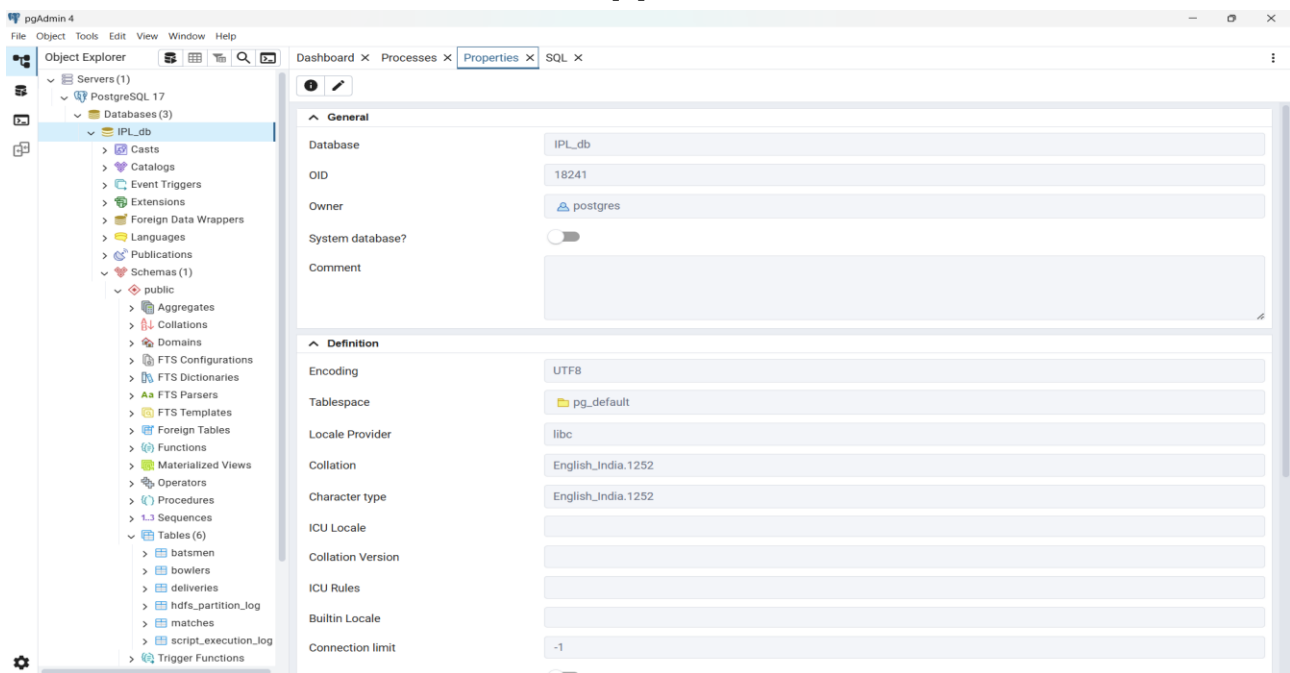


Fig 5.1. Database: IPL_db

SQL query for creating database:-

```
CREATE DATABASE "IPL_db"
WITH
OWNER = postgres
ENCODING = 'UTF8'
LC_COLLATE = 'English_India.1252'
LC_CTYPE = 'English_India.1252'
LOCALE_PROVIDER = 'libc'
TABLESPACE = pg_default
CONNECTION LIMIT = -1
IS_TEMPLATE = False;
```

5.2 Table Creation

1. batsmen

This table stores player-level batting data, including total runs, balls faced, strike rate, and dismissal information.

SQL command used to create batsmen table:-

```
CREATE TABLE batsmen (  
    batsmen_id integer primary key,  
    batsmen_name text,  
    total_runs integer,  
    total_matches integer,  
    total_innings integer,  
    total_not_outs integer,  
    average integer,  
    total_balls_faced integer,  
    strike_rate double precision,  
    hundreds integer,  
    fifties integer,  
    fours integer,  
    sixes integer,  
    partition_name text,  
    load_date date  
);
```

	column_name name	data_type character varying
1	batsmen_id	integer
2	batsmen_name	text
3	total_runs	integer
4	total_matches	integer
5	total_innings	integer
6	total_not_outs	integer
7	average	integer
8	total_balls_faced	integer
9	strike_rate	double precision
10	hundreds	integer
11	fifties	integer
12	fours	integer
13	sixes	integer
14	partition_name	text
15	load_date	date

Fig 5.2. batsmen table

2. bowlers

The **bowlers** table captures bowling statistics such as wickets taken, overs bowled, economy rate, and runs conceded. This helps in analyzing match-by-match and season-wide bowling performances.

SQL command used to create bowlers table:-

```
CREATE TABLE bowlers (  
    bowler_id integer primary key,  
    bowler_name text,  
    total_wkts integer,  
    total_matches integer,  
    total_inns integer,  
    total_overs numeric,  
    runs integer,  
    bowling_avg numeric,  
    economy_rate numeric,  
    strike_rate numeric,  
    total_4w integer,  
    total_5w integer,  
    partition_name text,  
    load_date date  
);
```

	column_name name	data_type character varying
1	bowler_id	integer
2	bowler_name	text
3	total_wkts	integer
4	total_matches	integer
5	total_inns	integer
6	total_overs	numeric
7	runs	integer
8	bowling_avg	numeric
9	economy_rate	numeric
10	strike_rate	numeric
11	total_4w	integer
12	total_5w	integer
13	partition_name	text
14	load_date	date

Fig 5.3. bowlers table

3. matches

The **matches** table contains metadata for each IPL match, including match ID, season, venue, teams, and result. It serves as the central table that connects the batsmen, bowlers, and deliveries datasets.

SQL command used to create matches table:-

```
CREATE TABLE matches (  
    match_id integer primary key,  
    city text,  
    date text,  
    match_type text,  
    player_of_match text,  
    batsmen_id integer,  
    bowlers_id integer,  
    venue text,  
    team1 text,  
    team2 text,  
    toss_winner text,  
    toss_decision text,  
    winner text,  
    result text,  
    result_margin integer,  
    target_runs integer,  
    target_overs integer,  
    super_over text,  
    method text,  
    umpire1 text,  
    umpire2 text,  
    season text,  
    rained text,  
    partition_name text  
);
```

	column_name 	data_type 
1	match_id	integer
2	city	text
3	date	text
4	match_type	text
5	player_of_match	text
6	batsmen_id	integer
7	bowlers_id	integer
8	venue	text
9	team1	text
10	team2	text
11	toss_winner	text
12	toss_decision	text
13	winner	text
14	result	text
15	result_margin	integer
16	target_runs	integer
17	target_overs	integer
18	super_over	text
19	method	text
20	umpire1	text
21	umpire2	text
22	season	text
23	rained	text
24	partition_name	text
25	load_date	date

Fig 5.4. matches table

4. deliveries

The **deliveries** table records ball-by-ball level data such as batsman name, bowler name, runs scored, extras, and wickets. It forms the most granular layer of the dataset and acts as the primary source for aggregating statistics in the batsmen and bowlers tables.

SQL command used to create deliveries table:


```
CREATE TABLE deliveries (
    match_id      integer,
    inning         integer,
    batting_team   text,
    bowling_team   text,
    over           integer,
    ball           integer,
    batter         text,
    bowler         text,
    non_striker    text,
    batsman_runs   integer,
    extra_runs     integer,
    total_runs     integer,
    extras_type    text,
    is_wicket      integer,
    player_dismissed text,
    dismissal_kind text,
    fielder        text,
    runs_per_over  bigint,
    partition_name text,
    load_date      date
);
```

	column_name 	data_type 
1	match_id	integer
2	inning	integer
3	batting_team	text
4	bowling_team	text
5	over	integer
6	ball	integer
7	batter	text
8	bowler	text
9	non_striker	text
10	batsman_runs	integer
11	extra_runs	integer
12	total_runs	integer
13	extras_type	text
14	is_wicket	integer
15	player_dismissed	text
16	dismissal_kind	text
17	fielder	text
18	runs_per_over	bigint
19	partition_name	text
20	load_date	date

Fig 5.5. deliveries table

5. hdfs_partition_log

The **hdfs_partition_log** table tracks every new HDFS partition created during the ETL pipeline runs. Each partition corresponds to a batch of processed data and includes timestamps, table names, and partition paths. This table is useful for data lineage and rollback operations.

SQL command used to create hdfs_partition_log table:-

```
CREATE TABLE hdfs_partition_log (
    id      integer,
    table_name text,
    partition_name text,
    hdfs_path text,
    date     date
);
```

	column_name 	data_type 
1	id	integer
2	table_name	text
3	partition_name	text
4	hdfs_path	text
5	date	date

Fig 5.6. hdfs_partition_log table

6. script_execution_log

This table maintains execution details of PySpark scripts triggered through Airflow DAGs. It stores metadata such as start and end time, run time, and status (success or failure). This log supports both operational monitoring and audit trail generation.

SQL command used to create script_execution_log table:-

```
CREATE TABLE script_execution_log (  
    id          serial,  
    script_name  character varying,  
    execution_date date,  
    start_time   timestamp without  
time zone,  
    end_time     timestamp without  
time zone,  
    run_time     interval,  
    data_loaded  character varying,  
    status       character varying  
);
```

	column_name 	data_type 
	name	character varying
1	id	integer
2	script_name	character varying
3	execution_date	date
4	start_time	timestamp without time zone
5	end_time	timestamp without time zone
6	run_time	interval
7	data_loaded	character varying
8	status	character varying

Fig 5.7. script_execution_log

5.3 Chapter Summary

Each of the above tables is automatically updated through PySpark scripts orchestrated by Apache Airflow. The **script_execution_log** and **hdfs_partition_log** tables provide essential transparency in pipeline execution and data movement. The analytical tables (**batsmen**, **bowlers**, **matches**, and **deliveries**) are connected to **Power BI** to visualize trends, player performance metrics, and match statistics in real time.

Chapter 6

Apache Hadoop

Apache Hadoop is an open-source framework designed for distributed storage and processing of large datasets across clusters of computers. It provides a reliable, scalable, and fault-tolerant way to manage big data. Hadoop works on the principle of dividing large files into smaller blocks and distributing them across multiple nodes for parallel processing. Its core components, **HDFS (Hadoop Distributed File System)** and **YARN (Yet Another Resource Negotiator)**, form the backbone for big data storage and resource management.

In this project, Hadoop serves as the **landing and storage layer** where all raw IPL data is first stored before being processed by PySpark. Using HDFS ensures that the data remains highly available and can be accessed efficiently during each ETL cycle. [1]

6.1 Benefits of Hadoop

Using Hadoop in this pipeline offered several operational and performance advantages:

- **Scalability:** Hadoop can easily handle growing data volumes by adding more nodes to the cluster.
- **Fault Tolerance:** Data is replicated across multiple nodes, ensuring that it remains accessible even if one node fails.
- **Cost Efficiency:** Since Hadoop can run on commodity hardware, it significantly reduces infrastructure costs.
- **Parallel Processing:** Data blocks are processed simultaneously across the cluster, improving processing speed.
- **Integration Flexibility:** Hadoop integrates seamlessly with PySpark, Kafka, and Airflow, making it an ideal foundation for a modern data engineering pipeline. [1]

6.2 Hadoop as the Landing Server

In the architecture of this project, Hadoop is represented as the **Landing Server**, the first point where raw data enters the system. All CSV files containing IPL data, such as batsmen, bowlers, matches, and deliveries are stored in the HDFS before any transformation or processing takes place.

This setup ensures data consistency, provides a historical record of all ingested data, and enables easy partitioning for each ETL run. PySpark scripts read directly from these HDFS locations, transform the data, and then load the cleaned version into PostgreSQL.

```
/files
|-batsmen
    |-batsmen_file
|-bowlers
    |-bowlers_file
|-matches
    |-matches_files
|-deliveries
    |-deliveries_files
/archives
|-batsmen
    |-batsmen_file
|-bowlers
    |-bowlers_file
|-matches
    |-matches_files
|-deliveries
    |-deliveries_files
/partition
|-partition_files
```

Fig 6.1. Folder Structure

Files received in the /files/batsmen, /files/bowlers, /files/matches and /files/deliveries, these are the landing directories. PySpark read files from these directories and then do the tasks.

After the data loaded into the PostgreSQL table, all files move from landing directory to archives directories.

6.3 Folder Structure and Partitioning

A structured folder hierarchy was created within HDFS to maintain organized data flow and simplify pipeline automation. The directory structure followed a modular pattern for each table:

Each table directory contains subfolders for dynamically created **partitions**, generated during each ETL run. The naming convention used was:

tablename_YYYYMMDD_HHMMSS

For example:

/data/ipl/batsmen/batsmen_20251007_153020

These partitions are automatically logged into the **hdfs_partition_log** table in PostgreSQL through the PySpark scripts. Once the data is processed and successfully loaded into the database, the source files are moved to the **archive** directory to prevent reprocessing in future runs.

6.4 Chapter Summary

Apache Hadoop plays a critical role in this project's architecture by serving as the central data landing zone. Its distributed storage, high availability, and efficient integration with PySpark made it an ideal choice for managing IPL datasets. The structured HDFS folder hierarchy and partitioning mechanism also enhanced the traceability and auditability of the ETL process.

Chapter 7

PySpark (Python API for Apache Spark)

PySpark is the Python API for Apache Spark, a powerful distributed computing framework designed for processing large-scale data efficiently. It combines the simplicity of Python with the scalability and performance of Spark, allowing developers to handle massive datasets that traditional Python libraries like Pandas cannot manage.

In this project, PySpark serves as the **data transformation and processing layer**. It reads raw IPL data from Hadoop Distributed File System (HDFS), performs data cleaning and transformation, and then writes the structured and validated data into PostgreSQL. By using PySpark, the ETL pipeline achieves automation, scalability, and fault tolerance, all essential features for handling real-world datasets. [2]

7.1 Why PySpark is Used

PySpark was chosen for this project due to its distributed architecture and flexibility for large-scale data processing. Some key reasons include:

- **Distributed Processing:** PySpark divides data across multiple nodes, allowing parallel execution of transformations.
- **Scalability:** It can handle datasets ranging from a few megabytes to several terabytes without code changes.
- **Integration Support:** PySpark connects seamlessly with Hadoop, PostgreSQL, and Airflow.
- **In-memory Computation:** It stores data in memory during transformations, reducing I/O and improving performance.
- **Resilience:** In case of task failures, Spark automatically retries and continues from checkpoints, ensuring fault tolerance. [2]

7.2 PySpark vs Pandas

Although both PySpark and Pandas are used for data analysis and transformation, the two differ in scalability and performance. Pandas operates on a single machine, making it suitable for smaller datasets. In contrast, PySpark is built for distributed environments and can handle data far beyond a single system's memory. [2]

Aspect	Pandas	PySpark
Processing Model	Single-node (runs on one machine)	Distributed (runs on multiple nodes)
Data Volume	Best for small to medium datasets	Designed for very large datasets
Performance	Slower for big data due to limited memory	Faster due to parallel and in-memory processing
Integration	Limited to local data files	Integrates with Hadoop, HDFS, PostgreSQL, and more
Fault Tolerance	No fault recovery	Automatic recovery from node failures

Table 7.1. PySpark vs Pandas

In this project, **PySpark was preferred over Pandas** because IPL datasets include millions of records at the delivery level, making distributed processing essential. Pandas would have struggled with memory constraints, while PySpark efficiently processed and transferred data between HDFS and PostgreSQL.

7.3 PySpark Scripts

A total of **four PySpark scripts** were developed as part of this project. Each script focused on a specific dataset and followed a structured flow:

1. Read raw data from HDFS.
2. Apply data validation and transformation.
3. Load cleaned data into PostgreSQL.
4. Log execution details in PostgreSQL tables for traceability.

1. batsmen.py

This script processes the **Batsmen** dataset, performing data validation and calculation of performance metrics. It ensures that fields such as total runs, matches, and strike rates are correctly formatted and non-null. The cleaned data is then inserted into the **batsmen** table in PostgreSQL.

Key Operations:

- Read raw data from HDFS.
- Handle missing or incorrect data.
- Calculate batting averages, strike rates, and boundary counts.
- Write transformed data to PostgreSQL.
- Log execution and partition information.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import trim, col, lit, current_date, to_date
from datetime import datetime
import subprocess
import psycpg2
import traceback
import sys

# --- Start Timer ---
script_name = "batsmen_script"
start_time = datetime.now()
execution_date = start_time.date()
status = "Started"
data_loaded = "No" # Default to No

try:
    # Generate Partition Name
    table_name = "batsmen"
    timestamp = start_time.strftime("%Y%m%d_%H%M%S")
    partition_name = f"{table_name}_{timestamp}"
    hdfs_partition_path = f"/partitions/{partition_name}"

    # Initiate Spark session
    spark = SparkSession.builder.appName("batsmen.csv script").getOrCreate()

    # Read CSV from HDFS
    batsmen = spark.read.csv('hdfs://localhost:9000/files/batsmen/batsmen_*.csv', sep=',', header=True, inferSchema=True)

    # Transformations
    batsmen = batsmen.withColumn("average", col("average").try_cast("int"))
    batsmen = batsmen.select([trim(col(c)).alias(c) if batsmen.schema[c].dataType.simpleString() == 'string' else col(c) for c in batsmen.columns])

    # Add New Columnn
    batsmen = batsmen.withColumn("partition_name", lit(partition_name))
    batsmen = batsmen.withColumn("load_date", to_date(current_date()))

    # Write to PostgreSQL
    batsmen.write.format("jdbc")\
        .option("url", "jdbc:postgresql://192.168.1.4:5432/IPL_db")\
        .option("driver", "org.postgresql.Driver")\
        .option("dbtable", "batsmen")\
```

Fig 7.1. batsmen.py script

2. bowlers.py

The **Bowlers** script focuses on calculating bowling metrics like total wickets, bowling average, and economy rate. It also ensures consistency between bowlers' data and match details. The final cleaned dataset is stored in the **bowlers** table in PostgreSQL.

Key Operations:

- Read data from HDFS.
- Compute economy and strike rates.
- Validate textual fields such as bowler names.
- Load transformed data into PostgreSQL.
- Update logs and partition metadata.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import trim, col, lit, current_date, to_date
from datetime import datetime
import subprocess
import psycog2
import traceback
import sys

# --- Start Timer ---
script_name = "bowlers_script"
start_time = datetime.now()
execution_date = start_time.date()
status = "Started"
data_loaded = "No" # Default to No

try:
    # Generate Partition Name
    table_name = "bowlers"
    timestamp = start_time.strftime("%Y%m%d_%H%M%S")
    partition_name = f"{table_name}_{timestamp}"
    hdfs_partition_path = f"/partitions/{partition_name}"

    # Initiate Spark session
    spark = SparkSession.builder.appName("bowlers.csv script").getOrCreate()

    # Read CSV from HDFS
    bowlers = spark.read.csv('hdfs://localhost:9000/files/bowlers/bowlers_*.csv', sep=',', header=True, inferSchema=True)

    # Transformations
    bowlers = bowlers.withColumn("bowling_avg", col("bowling_avg").try_cast("numeric"))
    bowlers = bowlers.withColumn("total_overs", col("total_overs").try_cast("numeric"))
    bowlers = bowlers.withColumn("economy_rate", col("economy_rate").try_cast("numeric"))
    bowlers = bowlers.withColumn("strike_rate", col("strike_rate").try_cast("numeric"))
    bowlers = bowlers.select([trim(col(c)).alias(c) if bowlers.schema[c].dataType.simpleString() == 'string' else col(c) for c in bowlers.columns])

    # Add New Column
    bowlers = bowlers.withColumn("partition_name", lit(partition_name))
    bowlers = bowlers.withColumn("load_date", to_date(current_date()))

    # Write to PostgreSQL
    bowlers.write.format("jdbc")\
```

Fig 7.2. bowlers.py script

3. matches.py

This script manages **match-level** data, establishing connections between players and matches using foreign key references. It ensures uniformity across multiple datasets and captures essential details such as venue, toss decision, result, and winner.

Key Operations:

- Load match data from HDFS.
- Standardize date formats and text fields.
- Map batsmen and bowlers through IDs.
- Store results in the matches table.
- Record ETL run details in logs.

```

from pyspark.sql import SparkSession
import subprocess
from pyspark.sql.functions import when,trim,col,substring,lit, current_date, to_date
from datetime import datetime
import psycopg2
import traceback
import sys

# --- Start Timer ---
script_name = "matches_script"
start_time = datetime.now()
execution_date = start_time.date()
status = "Started"
data_loaded = "No"

try:
    #Generate Partition Name
    table_name = "matches"
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    partition_name = f"{table_name}_{timestamp}"
    hdfs_partition_path = f"/partitions/{partition_name}"

    #initiate spark session
    spark = SparkSession.builder \
        .appName("Matches.csv script") \
        .getOrCreate()

    #read file from hdfs
    matches=spark.read.csv('hdfs://localhost:9000/files/matches/matches_*.csv',sep=',',header=True,inferSchema=True)

    #extract year from date column and create a new column
    matches=matches.withColumn('season',substring("date",7,4))

    #create a new column with condition
    matches=matches.withColumn("rained",when(col("method")=="D/L","rain").otherwise(None))

    #trim extra spaces from start and end
    matches=matches.select([ trim(col(c)).alias(c) if matches.schema[c].dataType.simpleString() == 'string' else col(c)
        for c in matches.columns
    ])

```

Fig 7.3. matches.py scripts

4. deliveries.py

The **Deliveries** script processes granular ball-by-ball data. Given the dataset's size, this script demonstrates the full power of PySpark's distributed processing. It computes derived statistics such as total runs per over, wicket count, and dismissal type.

Key Operations:

- Read delivery-level data from HDFS.
- Perform aggregations like total runs and extras.
- Identify wickets and dismissal details.
- Load results into PostgreSQL.
- Maintain logs for every ETL execution.

```

from pyspark.sql import SparkSession
import subprocess
from pyspark.sql.functions import trim,col,sum,when,substring,lit, current_date, to_date
from pyspark.sql.window import Window
from pyspark.sql import functions as F
from datetime import datetime
import psycpg2
import traceback
import sys

# --- Start Timer ---
script_name = "deliveries_script"
start_time = datetime.now()
execution_date = start_time.date()
status = "Started"
data_loaded = "No"

try:
    #Generate Partition Name
    table_name = "deliveries"
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    partition_name = f"{table_name}_{timestamp}"
    hdfs_partition_path = f"/partitions/{partition_name}"

    #initiate spark session
    spark = SparkSession.builder \
        .appName("Deliveries.csv script") \
        .getOrCreate()

    #extract file from hdfs
    deliveries=spark.read.csv('hdfs://localhost:9000/files/deliveries/deliveries_*.csv',sep=',',header=True,inferSchema=True)

    #trim extra spaces from start and end
    deliveries=deliveries.select([ trim(col(c)).alias(c) if deliveries.schema[c].dataType.simpleString() == 'string' else col(c)
        for c in deliveries.columns
    ])

    #create new column using other columns
    window_spec=Window.partitionBy("match_id", "inning", "over")
    deliveries = deliveries.withColumn("runs_per_over", F.sum("total_runs").over(window_spec))

```

Fig 7.4. deliveries.py script

7.4 Chapter Summary

PySpark played a central role in the project's data pipeline, transforming raw data into a structured, analysis-ready format. Its distributed architecture provided scalability, while its integration with Hadoop, Airflow, and PostgreSQL created a seamless end-to-end data flow. Compared to Pandas, PySpark proved to be a more efficient and reliable choice for handling large IPL datasets, enabling faster data transformations and consistent ETL execution.

Chapter 8

Partitioning and Python Script

In a data engineering pipeline, **partitioning** plays a crucial role in improving performance, data organization, and manageability. It allows large datasets to be divided into smaller, logical segments, which makes processing and maintenance more efficient.

In this project, partitioning was implemented within the **Hadoop Distributed File System (HDFS)** to organize IPL data dynamically during each ETL run. A dedicated Python script named **monitoring.py** was also developed to automate the relationship between HDFS partitions and PostgreSQL tables, ensuring data consistency and synchronization. [1] [11]

8.1 Purpose of Partitioning

The IPL dataset includes millions of records across multiple tables such as **batsmen**, **bowlers**, **matches**, and **deliveries**. Managing such a large volume of data efficiently requires a structured approach to storage. Partitioning helped achieve this by creating separate folders in HDFS for each ETL execution. [1]

Each run generated a new partition folder with a unique name format:

tablename_YYYYMMDD_HHMMSS

For example:

batsmen_20251007_153020

matches_20251007_153020

This structure provided several benefits:

- **Improved Performance:** Reading and writing operations became faster because Spark and Hadoop processed only relevant partitions instead of scanning the entire dataset.
- **Data Traceability:** Each ETL execution created a new, timestamped partition, which made it easy to track when and how data was processed.
- **Simplified Maintenance:** If a specific data load had errors or needed reprocessing, only that partition could be modified without affecting other runs.
- **Enhanced Data Governance:** Each partition's metadata was stored in the PostgreSQL table **hdfs_partition_log**, maintaining a link between HDFS folders and database entries.

8.2 Linking Partitions with Database Tables

Each time the PySpark ETL scripts ran, they created new HDFS partitions for all four main tables:

- **batsmen**
- **bowlers**
- **matches**
- **deliveries**

Along with data transformation, each run also inserted a record into the **hdfs_partition_log** table in PostgreSQL, containing details such as:

- Partition name
- Table name
- HDFS path
- Date

This ensured that every HDFS partition had a corresponding entry in the database, establishing a one-to-one relationship between stored data and its processed version.

8.3 The partition_deletion.py Script

The **monitoring.py** script is a custom Python utility developed to maintain synchronization between the **HDFS partitions** and the **PostgreSQL database tables**. Its main objective is to ensure that if a partition folder is manually or automatically deleted from HDFS, the corresponding data in PostgreSQL is also deleted, preserving consistency across systems. Instead of writing separate monitoring scripts for each table, a single consolidated script, **monitoring.py** was developed to handle all four tables efficiently.

```
import subprocess
import psycopg2
import time

start = time.time()

# Step 1: Fetch all partitions from DB
conn = psycopg2.connect(
    host="192.168.1.4", dbname="IPL_db", user="postgres", password="password"
)
cur = conn.cursor()
cur.execute("SELECT partition_name FROM hdfs_partition_log")
db_partitions = set(row[0] for row in cur.fetchall())

# Step 2: List all folders under /partitions from HDFS (only 1 subprocess call)
hdfs_ls = subprocess.run(
    ["hdfs", "dfs", "-ls", "/partitions"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
)

# Parse existing partition folders
hdfs_partitions = set()
if hdfs_ls.returncode == 0:
    for line in hdfs_ls.stdout.splitlines():
        if "/partitions/" in line:
            parts = line.strip().split()
            if len(parts) == 8: # typical output format
                path = parts[-1]
                partition = path.split("/")[-1]
                hdfs_partitions.add(partition)
else:
    print("Error listing HDFS directory:", hdfs_ls.stderr)

#print(f"Total partitions in HDFS: {len(hdfs_partitions)}")

# Step 3: Find missing ones
missing_partitions = db_partitions - hdfs_partitions
print(f"Missing partitions: {missing_partitions}")
```

Fig 8.1. partition_deletion.py script

8.4 Working Principle of scripts

The script operates in a continuous or scheduled mode, depending on how it's triggered (manually or via Airflow). Its workflow can be summarized as follows:

1. **Scan HDFS Directories:**

The script connects to the Hadoop file system and scans all table directories under:
/files/partitions

It retrieves the list of existing partition folders for each table.

2. **Compare with Database Log:**

It queries the **hdfs_partition_log** table in PostgreSQL to fetch all partition entries that are marked as active.

3. **Detect Missing Partitions:**

If the script finds any partition listed in the database but missing in HDFS, it identifies those as deleted or unavailable.

4. **Delete Corresponding Data from PostgreSQL:**

For each missing partition, the script automatically deletes all records in the corresponding table (batsmen, bowlers, matches, deliveries and hdfs_partition_log) that belong to that partition.

This is achieved using SQL statements that reference the unique partition key or timestamp.

5. **Update Logs:**

After deletion, the script updates the hdfs_partition_log table to mark that partition as “deleted” and records the deletion timestamp.

The action is also appended to a daily log file for audit and troubleshooting.

8.5 Key Features

The **monitoring.py** script includes several important features:

- **Centralized Monitoring:**

One script manages all four main data tables, avoiding duplication and simplifying maintenance.

- **Automated Data Sync:**

It automatically ensures that database entries always reflect the current state of HDFS partitions.

- **Error Handling and Logging:**

Any missing partitions, failed deletions, or database errors are recorded in log files for review.

- **Performance Optimization:**

By checking partitions incrementally rather than scanning entire tables, the script reduces overhead and executes quickly.

- **Integration with Airflow:**

The script can be added as a separate Airflow DAG or scheduled task to run at fixed intervals, keeping both systems continuously synchronized.

8.6 Example Scenario

Suppose the HDFS partition **bowlers_20251007_153020** is manually deleted from Hadoop.

When partition_deletion.py runs, it performs the following steps:

1. Scans the /files/partitions directory.

2. Finds that bowlers_20251007_153020 no longer exists.
3. Checks PostgreSQL and finds a corresponding entry in hdfs_partition_log.
4. Deletes all related rows from the bowlers table that belong to that partition.
5. Updates the hdfs_partition_log to mark the partition as deleted.

This ensures that both the file system and database remain perfectly synchronized, preventing orphaned or outdated records.

8.7 Chapter Summary

Partitioning and automated monitoring form the foundation of reliable data lifecycle management in this project. The use of dynamic HDFS partitions ensured data organization, scalability, and easier maintenance, while the **partition_deletion.py** script provided a safeguard for data consistency. Together, they establish a self-managing system where each data load is isolated, traceable, and automatically cleaned when its corresponding partition is removed, ensuring a consistent and well-governed data environment.

Chapter 9

Apache Airflow and Email Alerts

Apache Airflow is an open-source workflow orchestration tool designed to programmatically author, schedule, and monitor workflows. It allows data engineers to define complex ETL pipelines as code, making workflows easier to manage, scale, and track. Airflow's core concept is the **DAG (Directed Acyclic Graph)**, which ensures tasks execute in the correct order without cycles, making it ideal for sequential and dependent tasks. [4]

9.1 Benefits of Airflow

1. **Automation and Scheduling:** Airflow allows tasks to run automatically based on schedules or triggers, reducing manual intervention.
2. **Scalability:** It supports parallel execution and distributed task processing, allowing pipelines to handle large volumes of data.
3. **Monitoring and Logging:** A web-based UI provides detailed logs for tasks, including successes, failures, retries, and execution times, giving full visibility into pipeline operations.
4. **Flexibility and Extensibility:** Workflows are defined in Python code, enabling conditional logic, dynamic task generation, and integration with tools like PySpark, Hadoop, and PostgreSQL. [4]

9.2 DAGs Explanation

A **DAG (Directed Acyclic Graph)** represents a workflow in Airflow. Each node is a task, and edges define dependencies between tasks. DAGs allow you to automate the execution of scripts, ensure dependencies are respected, and handle scheduling. [4]

In our project, two DAGs are set up:

1. **IPL_db.py DAG**
 - **Purpose:** Executes PySpark scripts serially for IPL-related tables (players, matches, deliveries).
 - **Workflow:**
 - Before running each PySpark script, the DAG checks the corresponding HDFS folder for new files.
 - If files exist, the script runs; if not, the task is skipped.
 - **Use case:** Efficient ETL processing that avoids running scripts unnecessarily and ensures only new data is ingested.

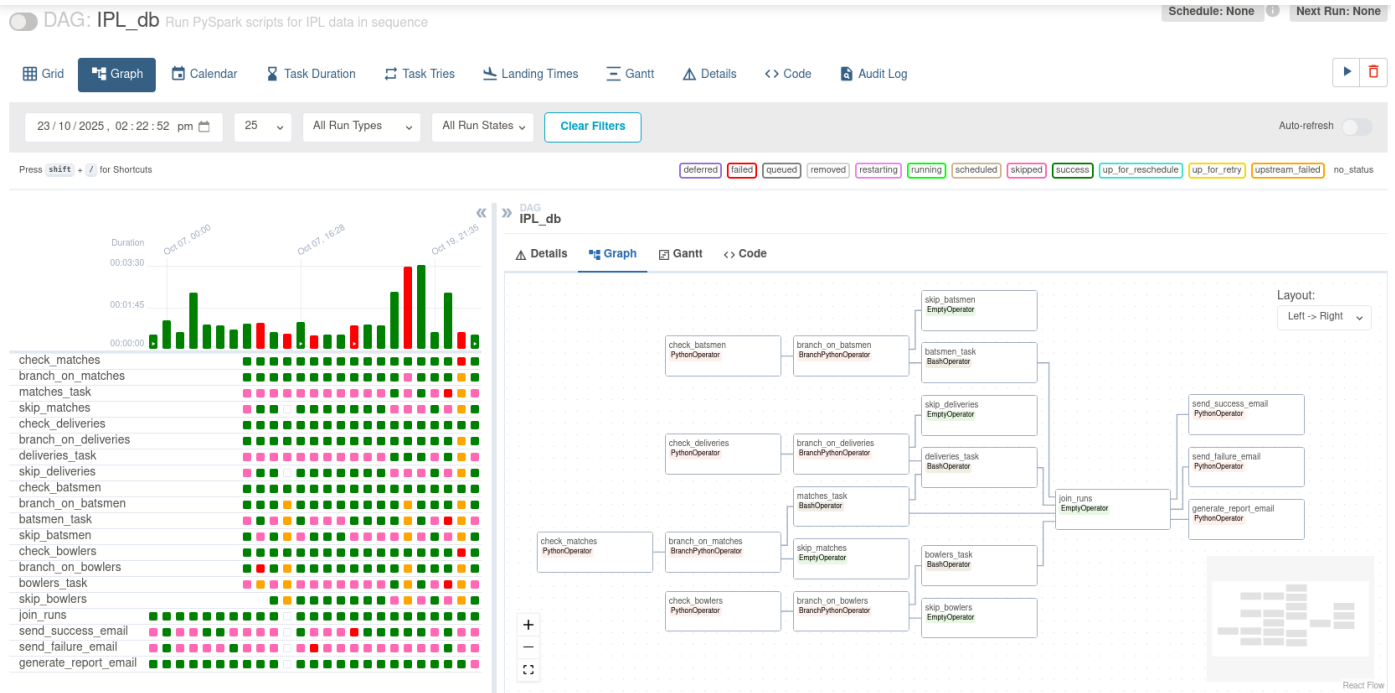


Fig 9.1. IPL_db dag

2. Partition_deletion.py DAG

- **Purpose:** Monitors HDFS partitions and related PostgreSQL tables every minute.
- **Workflow:**
 - The DAG runs a Python script that scans for deleted partitions in HDFS.
 - If a partition is deleted, corresponding rows in PostgreSQL tables are automatically removed.
- **Use case:** Maintains near real-time consistency between HDFS partitions and relational tables.

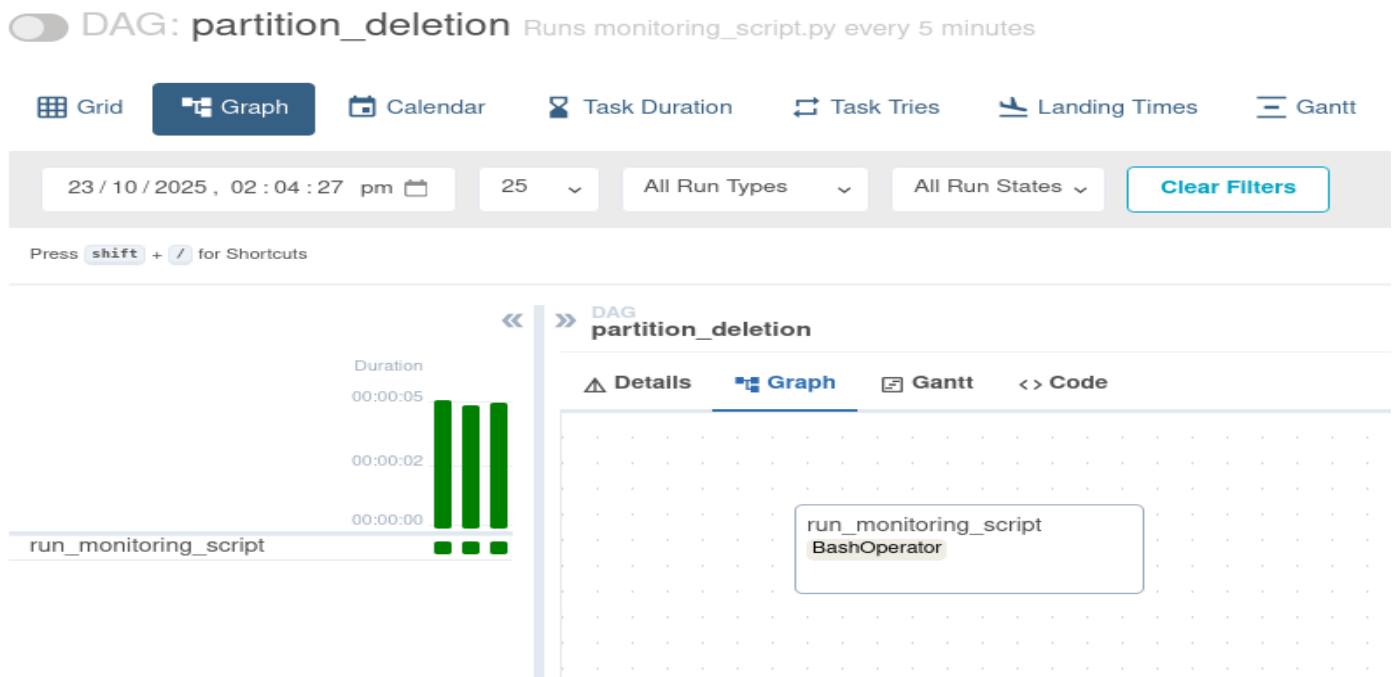


Fig 9.2. partition_deletion dag

9.3 Mailing Alerts

Airflow can send email notifications on task success, failure, or count report. Alerts can include logs, summary information, or metrics like inserted row counts. [12]

- **Success Emails:** After IPL_db.py completes, a summary email can include details like the number of rows processed for each table.
- **Failure Emails:** If a task fails (e.g., a PySpark script or the monitoring script), Airflow sends a failure email to notify the team.
- **Custom Content:** Emails can be customized with Python to include logs, table summaries, or other relevant data.

In my setup:

- IPL_db.py ensures tasks only run when files are present and sends success/failure emails for processed tables.
- partition_deletion.py ensures deleted HDFS partitions trigger corresponding updates in PostgreSQL and can send alerts if any action is required.

9.4 Chapter Summary

This chapter illustrates how Airflow orchestrates our ETL pipeline, automating PySpark workflows, monitoring HDFS partitions, and providing timely email notifications to maintain reliable, consistent data pipelines.

Chapter 10

PowerBI

Power BI is a business intelligence and data visualization tool developed by Microsoft. It allows users to transform raw data into interactive dashboards, reports, and charts, providing actionable insights for business and technical decision-making. Power BI connects to a wide range of data sources including databases (PostgreSQL, SQL Server), Excel, cloud services, and APIs.

It is widely used for analysing large datasets, creating visual representations of metrics, and sharing insights through interactive dashboards that update automatically based on underlying data. [8]

10.1. Benefits of Power BI

1. **Data Connectivity:** Power BI can connect to multiple data sources, both on-premises and cloud-based, allowing seamless integration of data from different systems.
2. **Interactive Visualizations:** Users can create dynamic dashboards with charts, graphs, maps, and tables that allow drill-down, filtering, and cross-highlighting.
3. **Automation:** Dashboards can refresh automatically when the data source updates, providing near real-time insights.
4. **Collaboration and Sharing:** Reports and dashboards can be shared across teams or published to Power BI Service, enabling collaborative decision-making.
5. **Self-Service Analytics:** Users without deep technical expertise can explore data, apply filters, and generate insights without relying entirely on IT teams. [8]

10.2. Use Case in Our Project

In our data engineering workflow, Power BI is used to:

- **Analyze IPL Data:** Connect to PostgreSQL where IPL data is stored after processing through PySpark.
- **Create Interactive Dashboards:** Display metrics such as match statistics, player performance, and team summaries.
- **Enable Drill-Down and Filtering:** Users can explore data by season, match, team, or player.

10.3. Chapter Summary

This chapter highlights how Power BI transforms processed IPL data into interactive dashboards, enabling visual analysis, real-time monitoring, and data-driven decision-making for stakeholder.

Chapter 11

End Results

The project aimed to build a complete data engineering pipeline for IPL data using PySpark, Hadoop, Airflow, and PostgreSQL, with visualization in Power BI. The end results demonstrate both the technical execution and business value of the pipeline.

11.1. Key Outputs of the Project

1. Automated ETL Pipeline

- PySpark scripts process IPL datasets (players, matches, deliveries) from raw CSV files stored in HDFS.
- The IPL_db.py DAG in Airflow ensures that scripts run serially, checking for the presence of new files before execution.
- Deleted HDFS partitions trigger automatic removal of corresponding rows in PostgreSQL via the partition_deletion.py DAG.

2. Partitioning and Data Consistency

- HDFS data is partitioned by table name and timestamp for efficient storage and retrieval.
- A single Python script monitors all partitions, ensuring that any deletion in HDFS is reflected in the relational database, maintaining consistency between distributed and structured storage.

3. Reliable Orchestration with Airflow

- Workflows are fully automated, reducing manual intervention.
- Tasks are monitored with detailed logs and email alerts for success or failure.
- The pipeline is scalable, allowing addition of new tables or scripts with minimal changes.

4. PostgreSQL Database Integration

- Processed and cleaned IPL data is stored in PostgreSQL for downstream analysis.
- Tables include accurate and updated records corresponding to HDFS partitions, ensuring relational integrity.

5. Real-Time Monitoring

- The partition_deletion.py DAG runs every minute to detect changes in HDFS and take appropriate actions in PostgreSQL.
- This ensures near real-time synchronization between HDFS and the relational database.

6. Interactive Visualization in Power BI

- Dashboards provide insights into match statistics, player performances, and team analytics.
- Filters and drill-down options allow users to explore data by match, season, player, or team.
- Dashboards refresh automatically using an On-Premises Data Gateway, reflecting the latest processed data.

7. Efficiency and Automation

- Scripts run only when new data is available, reducing unnecessary computation.

- Airflow orchestrates task dependencies and scheduling, enabling a fully automated and reliable ETL process.

8. Scalability and Extensibility

- The system can handle additional IPL seasons, new tables, or other datasets with minimal configuration.
- The pipeline design allows easy integration with cloud storage or additional BI tools in the future.

Below are the screenshots of tables and power bi which is the result of this project.

1. batsmen table: This table stores the cleaned and transformen data for batsmen.

1 `select * from batsmen;`

Data Output Messages Notifications

Showing rows: 1 to 668 Page No: 1 of 1

	batsmen_id integer	batsmen_name text	total_runs integer	total_matches integer	total_innings integer	total_not_outs integer	average integer	total_balls_faced integer	strike_rate double precision	hundreds integer	fifties integer
1	1	Aakash Chopra	53	7	6	0	[null]	71	69.33	0	0
2	2	Aaron Finch	2091	92	90	7	[null]	1631	119.69	0	15
3	3	AB de Villiers	4697	169	157	36	[null]	3048	147.29	2	37
4	4	Abdul Basith	1	1	1	1	0	1	100	0	0
5	5	Abdul Samad	577	50	40	10	[null]	395	131.2	0	0
6	6	Abdur Razzak	0	1	1	1	0	2	0	0	0
7	7	Abhijeet Tomar	4	1	1	0	4	8	50	0	0
8	8	Abhimanyu Mithun	34	14	8	3	[null]	26	120	0	0
9	9	Abhinav Manohar	231	19	15	1	[null]	174	112.53	0	0
10	10	Abhinav Mukund	19	3	2	0	[null]	22	45.24	0	0
11	11	Abhishek Jhunjunwala	217	21	16	3	[null]	210	86.03	0	1
12	12	Abhishek Nayar	672	60	50	12	[null]	577	107.82	0	0
13	13	Abhishek Raut	194	22	16	7	[null]	167	115.99	0	0
14	14	Abhishek Sharma	1377	63	61	7	[null]	887	147.09	0	7
15	15	Abishek Porel	360	18	16	2	[null]	236	132.98	0	2
16	16	Abu Nechim	14	7	4	3	5	6	300	0	0
17	17	Adam Gilchrist	2069	80	80	4	[null]	1495	137.35	2	11
18	18	Adam Milne	23	8	6	2	[null]	29	76.22	0	0
19	19	Adam Voges	181	9	7	3	[null]	143	126.57	0	0

Fig 11.1. Data Loaded in Batsmen Table

2. bowlers table: This table stores cleaned and transformed data for bowlers.

3 `select * from bowlers;`

Data Output Messages Notifications

Showing rows: 1 to 447 Page No: 1 of 1

	bowler_id [PK] integer	bowler_name text	total_wkts integer	total_matches integer	total_inns integer	total_overs numeric	runs integer	bowling_avg numeric	economy_rate numeric	strike_rate numeric	total_4w integer	total_5w integer
1	1	Aaron Finch	1	14	3	4	49	49	12	24	0	
2	2	Aavishkar Salvi	3	2	2	8	66	22	8	16	0	
3	3	Abdul Samad	1	12	3	7	96	96	14	42	0	
4	4	Abhimanyu thun	7	14	14	41	397	47	11	28	0	
5	5	Abhishek Jhunjunwala	1	5	3	6	38	38	6	36	0	
6	6	Abhishek Nayar	9	48	18	37	306	42	8	29	0	
7	7	Abhishek Sharma	11	46	23	41	351	34	9	23	0	
8	8	Abu Nechim	7	9	9	33	286	60	8	38	0	
9	9	Adam Ine	7	9	9	32	308	44	10	27	0	
10	10	Adam Zampa	29	20	20	70	558	26	8	19	0	
11	11	Adil Rashid	2	2	2	7	56	28	8	21	0	
12	12	Aditya Dole	5	3	3	11	112	22	10	13	0	
13	13	Aiden Markram	2	27	10	14	134	67	10	43	0	
14	14	Ajantha Mendis	8	10	10	40	285	34	7	29	0	
15	15	Ajit Agarkar	29	38	38	120	1059	40	9	26	0	
16	16	Ajit Chandila	11	12	12	39	242	22	6	21	1	
17	17	Akash Deep	7	8	8	27	319	52	13	25	0	
18	18	Akash Madhwal	19	13	13	43	424	28	10	16	1	
19	19	Akash Singh	5	6	6	19	188	38	10	23	0	

Fig 11.2. Data Loaded in Bowlers Table

3. matches table: This table stores cleaned and transformed data for matches.

5 `select * from matches;`

Data Output Messages Notifications

Showing rows: 1 to 816 Page No: 1 of 1

	match_id integer	city text	date text	match_type text	player_of_match text	batsmen_id integer	bowlers_id integer	venue text	team1 text
1	335982	Bangalore	18-04-2008	Match 1	BB McCullum	115	[null]	M Chinnaswamy Stadium	Royal Challengers Bangalo
2	335983	Chandigarh	19-04-2008	Match 2	MEK Hussey	351	[null]	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab
3	335984	Delhi	19-04-2008	Match 3	MF Maharoof	348	[null]	Feroz Shah Kotla	Delhi Daredevils
4	335985	Mumbai	20-04-2008	Match 4	MV Boucher	338	[null]	Wankhede Stadium	Mumbai Indians
5	335986	Kolkata	20-04-2008	Match 5	DJ Hussey	152	[null]	Eden Gardens	Kolkata Knight Riders
6	335987	Jaipur	21-04-2008	Match 6	SR Watson	529	[null]	Sawai Mansingh Stadium	Rajasthan Royals
7	335988	Hyderabad	22-04-2008	Match 7	V Sehwag	637	[null]	Rajiv Gandhi International Stadium, Uppal	Deccan Chargers
8	335989	Chennai	23-04-2008	Match 8	ML Hayden	343	[null]	MA Chidambaram Stadium, Chepauk	Chennai Super Kings
9	335990	Hyderabad	24-04-2008	Match 9	YK Pathan	658	[null]	Rajiv Gandhi International Stadium, Uppal	Deccan Chargers
10	335991	Chandigarh	25-04-2008	Match 10	KC Sangakkara	301	[null]	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab
11	335992	Bangalore	26-04-2008	Match 11	SR Watson	529	[null]	M Chinnaswamy Stadium	Royal Challengers Bangalo
12	335993	Chennai	26-04-2008	Match 12	JDP Oram	232	[null]	MA Chidambaram Stadium, Chepauk	Chennai Super Kings
13	335994	Mumbai	27-04-2008	Match 13	AC Gilchrist	17	[null]	Dr DY Patil Sports Academy	Mumbai Indians
14	335995	Chandigarh	27-04-2008	Match 14	SM Katich	563	[null]	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab
15	335996	Bangalore	28-04-2008	Match 15	MS Dhoni	375	[null]	M Chinnaswamy Stadium	Royal Challengers Bangalo
16	335997	Kolkata	29-04-2008	Match 16	ST Jayasuriya	[null]	336	Eden Gardens	Kolkata Knight Riders
17	335998	Delhi	30-04-2008	Match 17	GD McGrath	[null]	125	Feroz Shah Kotla	Delhi Daredevils
18	335999	Hyderabad	01-05-2008	Match 18	SE Marsh	533	[null]	Rajiv Gandhi International Stadium, Uppal	Deccan Chargers
19	336000	Jaipur	01-05-2008	Match 19	SA Asnodkar	587	[null]	Sawai Mansingh Stadium	Rajasthan Royals

Fig 11.3. Data Loaded in Matches Table

4. deliveries table: This table stores cleaned and transformed data for deliveries.

7 `select * from deliveries;`

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 783

	match_id integer	inning integer	batting_team text	bowling_team text	over integer	ball integer	batter text	bowler text	non_striker text	batsman_runs integer	extra_ru integer
1	1178407	1	Kings XI Punjab	Rajasthan Royals	5	5	MA Agarwal	JC Archer	KL Rahul		1
2	1178407	1	Kings XI Punjab	Rajasthan Royals	5	6	KL Rahul	JC Archer	MA Agarwal		0
3	1178407	1	Kings XI Punjab	Rajasthan Royals	6	1	MA Agarwal	IS Sodhi	KL Rahul		1
4	1178407	1	Kings XI Punjab	Rajasthan Royals	6	2	KL Rahul	IS Sodhi	MA Agarwal		1
5	1178407	1	Kings XI Punjab	Rajasthan Royals	6	3	MA Agarwal	IS Sodhi	KL Rahul		2
6	1178407	1	Kings XI Punjab	Rajasthan Royals	6	4	MA Agarwal	IS Sodhi	KL Rahul		6
7	1178407	1	Kings XI Punjab	Rajasthan Royals	6	5	MA Agarwal	IS Sodhi	KL Rahul		1
8	1178407	1	Kings XI Punjab	Rajasthan Royals	6	6	KL Rahul	IS Sodhi	MA Agarwal		0
9	1178407	1	Kings XI Punjab	Rajasthan Royals	7	1	MA Agarwal	S Gopal	KL Rahul		1
10	1178407	1	Kings XI Punjab	Rajasthan Royals	7	2	KL Rahul	S Gopal	MA Agarwal		1
11	1178407	1	Kings XI Punjab	Rajasthan Royals	7	3	MA Agarwal	S Gopal	KL Rahul		4
12	1178407	1	Kings XI Punjab	Rajasthan Royals	7	4	MA Agarwal	S Gopal	KL Rahul		6
13	1178407	1	Kings XI Punjab	Rajasthan Royals	7	5	MA Agarwal	S Gopal	KL Rahul		1
14	1178407	1	Kings XI Punjab	Rajasthan Royals	7	6	KL Rahul	S Gopal	MA Agarwal		0
15	1178407	1	Kings XI Punjab	Rajasthan Royals	8	1	MA Agarwal	IS Sodhi	KL Rahul		1
16	1178407	1	Kings XI Punjab	Rajasthan Royals	8	2	KL Rahul	IS Sodhi	MA Agarwal		1
17	1178407	1	Kings XI Punjab	Rajasthan Royals	8	3	MA Agarwal	IS Sodhi	KL Rahul		2
18	1178407	1	Kings XI Punjab	Rajasthan Royals	8	4	MA Agarwal	IS Sodhi	KL Rahul		0
19	1178407	1	Kings XI Punjab	Rajasthan Royals	8	5	KL Rahul	IS Sodhi	DA Miller		1

Fig 11.4. Data Loaded in Deliveries Table

5. HDFS Partition: Partitions are stored in HDFS location after each successful run.

```
[hadoop@vbox ~]$ hdfs dfs -ls /partitions
Found 6 items
drwxr-xr-x - hadoop supergroup 0 2025-10-20 03:07 /partitions/batsmen_20251020_030636
drwxr-xr-x - hadoop supergroup 0 2025-10-20 03:07 /partitions/bowlers_20251020_030649
drwxr-xr-x - hadoop supergroup 0 2025-10-20 03:08 /partitions/deliveries_20251020_030637
drwxr-xr-x - hadoop supergroup 0 2025-10-20 03:08 /partitions/deliveries_20251020_030640
drwxr-xr-x - hadoop supergroup 0 2025-10-20 03:25 /partitions/deliveries_20251020_032409
drwxr-xr-x - hadoop supergroup 0 2025-10-20 03:08 /partitions/matches_20251020_030651
[hadoop@vbox ~]$
```

Fig 11.5. Partition Stored in HDFS

6. hdfs_partition_log table: This table stores records for created partitions.

```
9 select * from hdfs_partition_log order by date desc;|
```

Data Output Messages Notifications

Showing

	id [PK] integer	table_name text	partition_name text	hdfs_path text	date date
1	202	batsmen	batsmen_20251020_030636	/partitions/batsmen_20251020_030636	2025-10-20
2	203	bowlers	bowlers_20251020_030649	/partitions/bowlers_20251020_030649	2025-10-20
3	204	matches	matches_20251020_030651	/partitions/matches_20251020_030651	2025-10-20
4	205	deliveries	deliveries_20251020_030637	/partitions/deliveries_20251020_030637	2025-10-20
5	206	deliveries	deliveries_20251020_030640	/partitions/deliveries_20251020_030640	2025-10-20
6	207	deliveries	deliveries_20251020_032409	/partitions/deliveries_20251020_032409	2025-10-20

Fig 11.6. Partition Logs Stored in `hdfs_partition_log` table

7. script_execution_log table: This table stores records of each run.

11

select * from script_execution_log order by execution_date desc;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 153

Page No: 1 of 1

	id [PK] integer	script_name character varying (100)	execution_date date	start_time timestamp without time zone	end_time timestamp without time zone	run_time interval	data_loaded character varying (10)	status character varying (20)
1	144	matches_script	2025-10-20	2025-10-20 02:12:10.462323	2025-10-20 02:12:24.341783	00:00:13.87946	Yes	Success
2	149	deliveries_script	2025-10-20	2025-10-20 03:06:40.700785	2025-10-20 03:08:35.56876	00:01:54.867975	Yes	Success
3	148	deliveries_script	2025-10-20	2025-10-20 03:06:37.777772	2025-10-20 03:08:33.57834	00:01:55.800568	Yes	Success
4	147	matches_script	2025-10-20	2025-10-20 03:06:51.089971	2025-10-20 03:08:11.491684	00:01:20.401713	Yes	Success
5	146	bowlers_script	2025-10-20	2025-10-20 03:06:49.184642	2025-10-20 03:08:03.620212	00:01:14.43557	Yes	Success
6	145	batsmen_script	2025-10-20	2025-10-20 03:06:36.96028	2025-10-20 03:07:54.490237	00:01:17.529957	Yes	Success
7	153	deliveries_script	2025-10-20	2025-10-20 03:24:09.721607	2025-10-20 03:25:14.253282	00:01:04.531675	Yes	Success
8	152	matches_script	2025-10-20	2025-10-20 03:24:04.643694	2025-10-20 03:24:53.764307	00:00:49.120613	No	Failed
9	151	batsmen_script	2025-10-20	2025-10-20 03:24:10.30289	2025-10-20 03:24:52.364076	00:00:42.061186	No	Failed
10	150	bowlers_script	2025-10-20	2025-10-20 03:23:56.632865	2025-10-20 03:24:41.090342	00:00:44.457477	No	Failed
11	142	matches_script	2025-10-17	2025-10-17 23:50:48.552786	2025-10-17 23:50:54.203793	00:00:05.651007	No	Failed
12	141	bowlers_script	2025-10-17	2025-10-17 23:32:36.729147	2025-10-17 23:32:48.604419	00:00:11.875272	Yes	Success
13	140	bowlers_script	2025-10-17	2025-10-17 23:30:41.008252	2025-10-17 23:30:53.710355	00:00:12.702103	Yes	Success
14	139	batsmen_script	2025-10-17	2025-10-17 22:21:32.276783	2025-10-17 22:21:43.278677	00:00:11.001894	Yes	Success
15	143	matches_script	2025-10-17	2025-10-17 23:54:07.630103	2025-10-17 23:54:19.632406	00:00:12.002303	Yes	Success
16	124	batsmen_script	2025-10-07	2025-10-07 06:02:26.987095	2025-10-07 06:02:46.78538	00:00:19.798285	Yes	Success
17	122	batsmen_script	2025-10-07	2025-10-07 05:38:57.793864	2025-10-07 05:39:17.584957	00:00:19.791093	Yes	Success
18	123	batsmen_script	2025-10-07	2025-10-07 05:50:26.892687	2025-10-07 05:50:44.019244	00:00:17.126557	Yes	Success
19	121	matches_script	2025-10-07	2025-10-07 05:26:41.603279	2025-10-07 05:26:53.961897	00:00:12.358618	Yes	Success

Fig 11.7. Script Execution Log Stored in Table

8. Successful Airflow Execution: Airflow Dag is successfully executed.

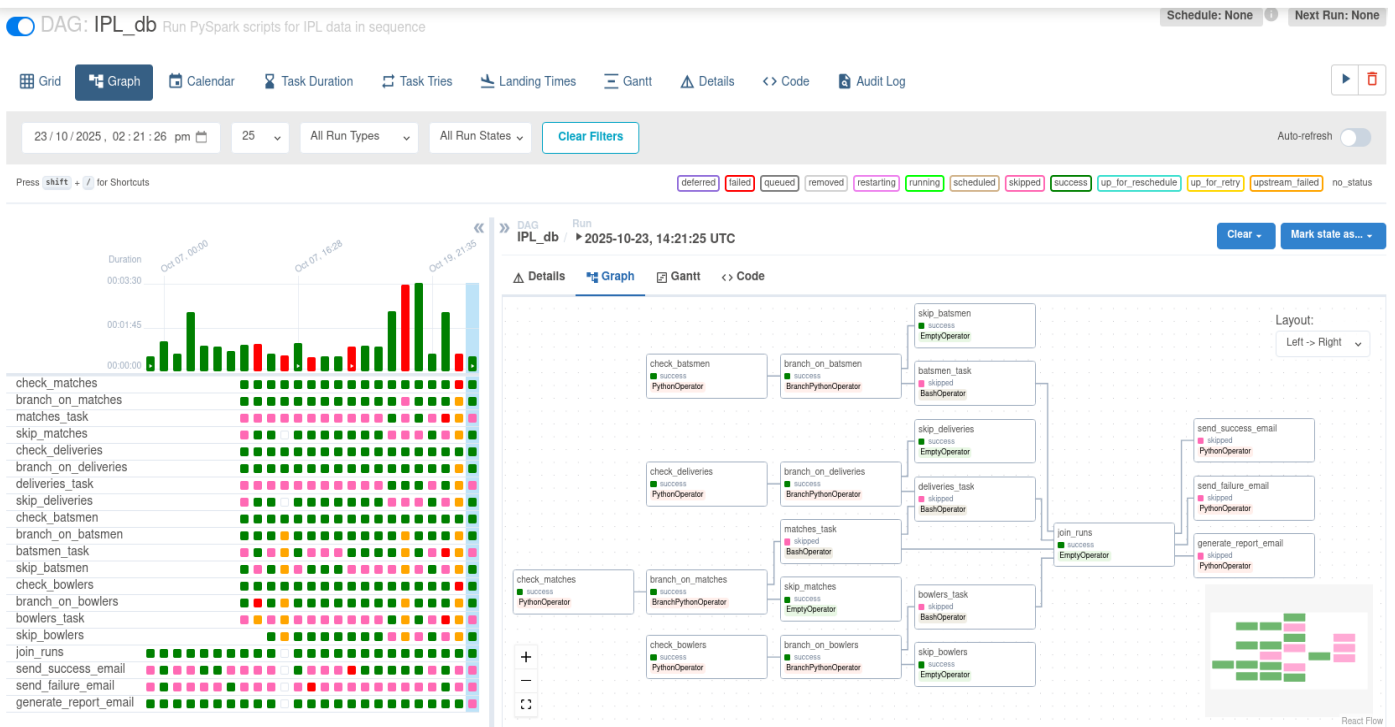


Fig 11.8. Successful Airflow IPL db dag

9. Success mail alert: Mail after successful script execution.

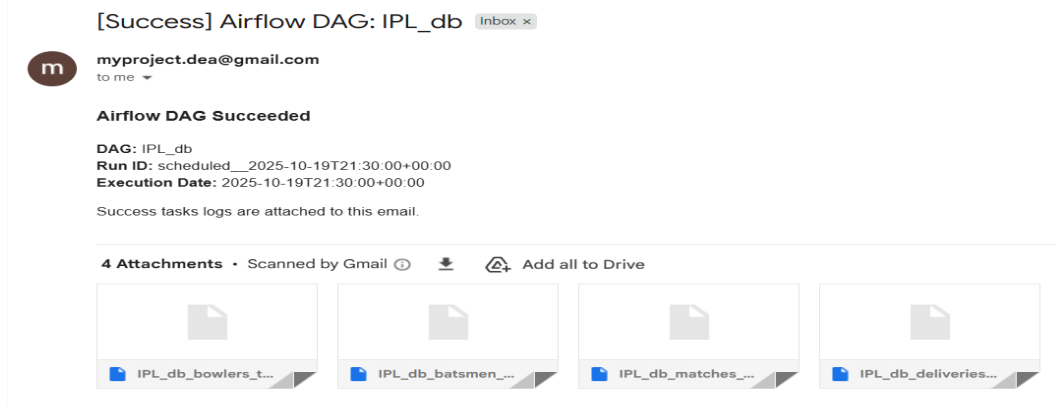


Fig 11.9. Success Mail with Logs

10. Failure mail alert: Mail if any script failed.

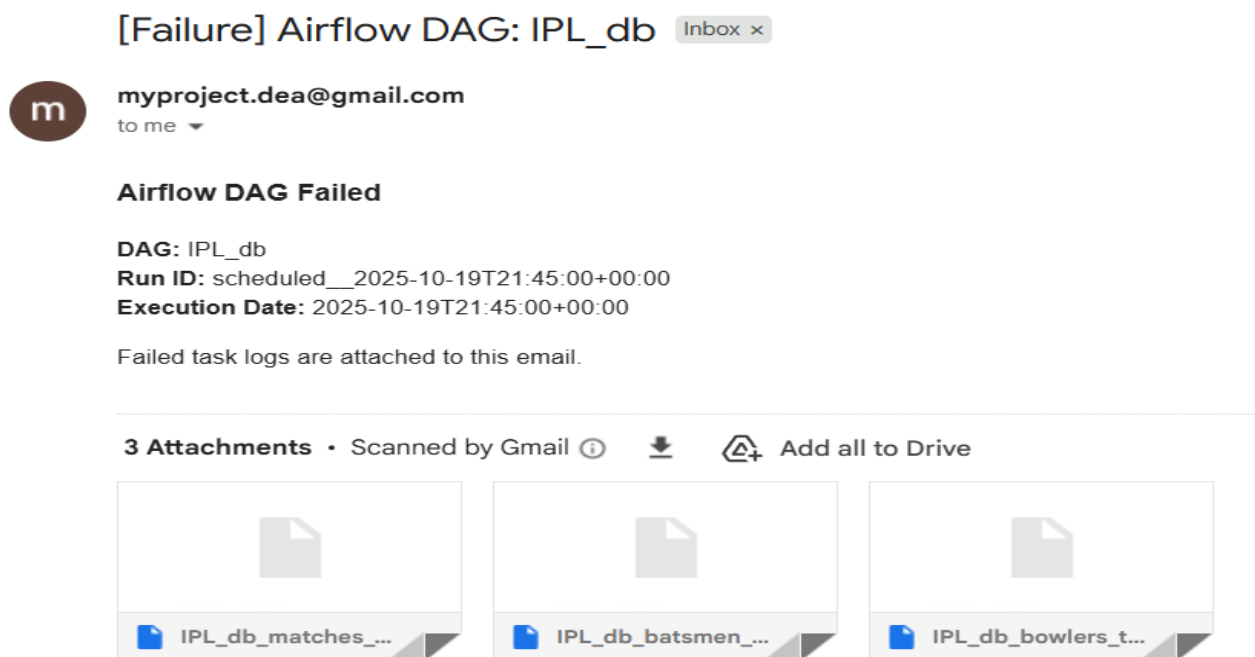


Fig 11.10. Failure Mail with Logs

11. Report Count mail alert: Report count after each successful run.

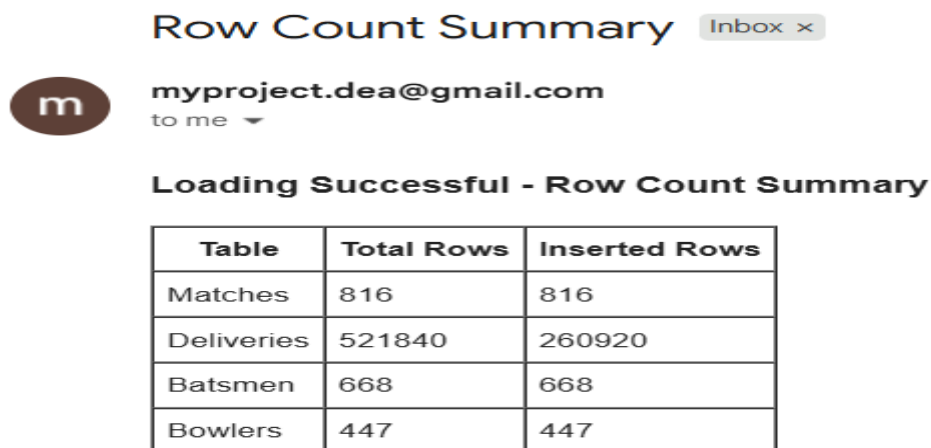


Fig 11.11. Count Report Mail

12. Dashboard: PowerBI dashboard for visualization.

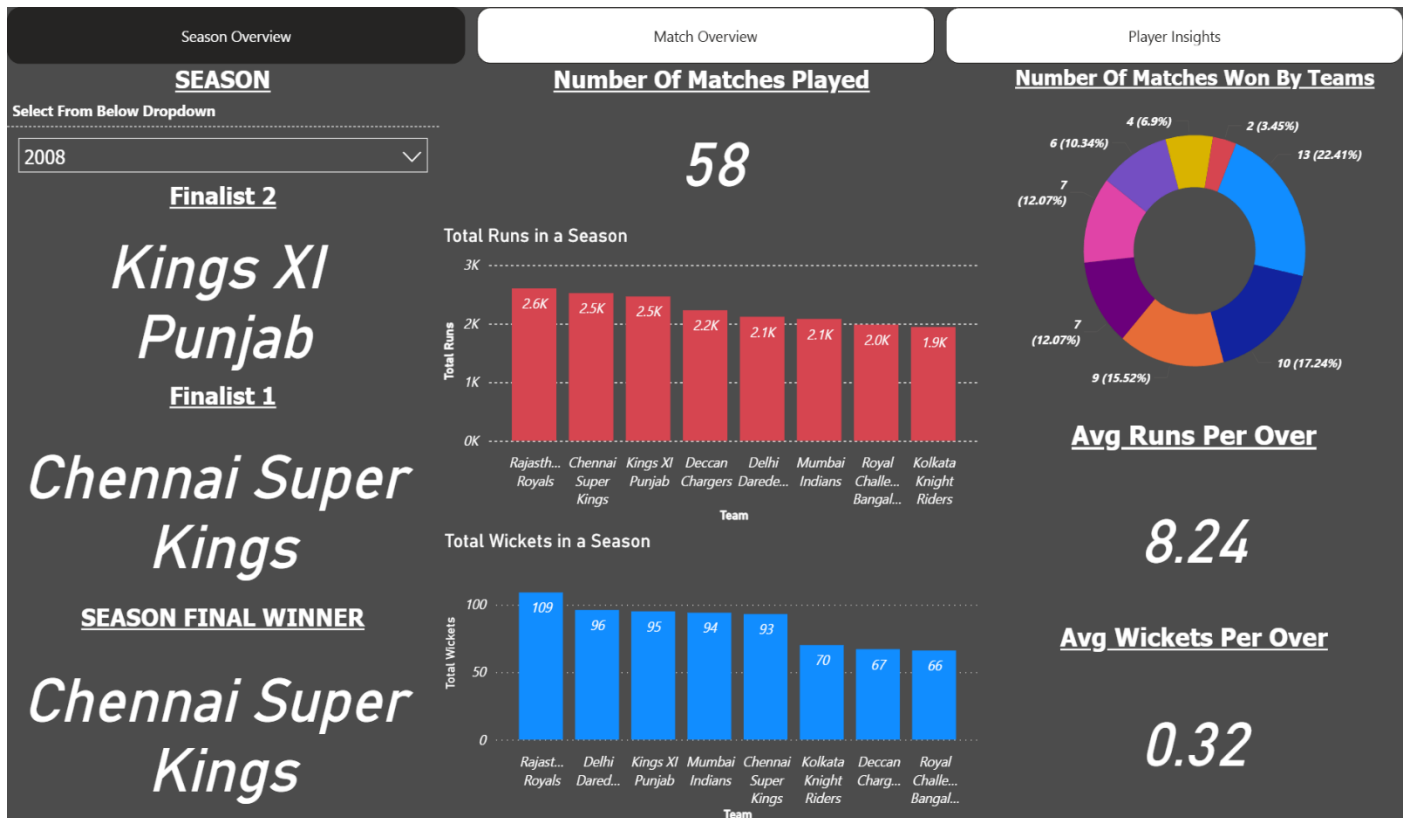


Fig 11.12. Power BI Dashboard Season Overview Page

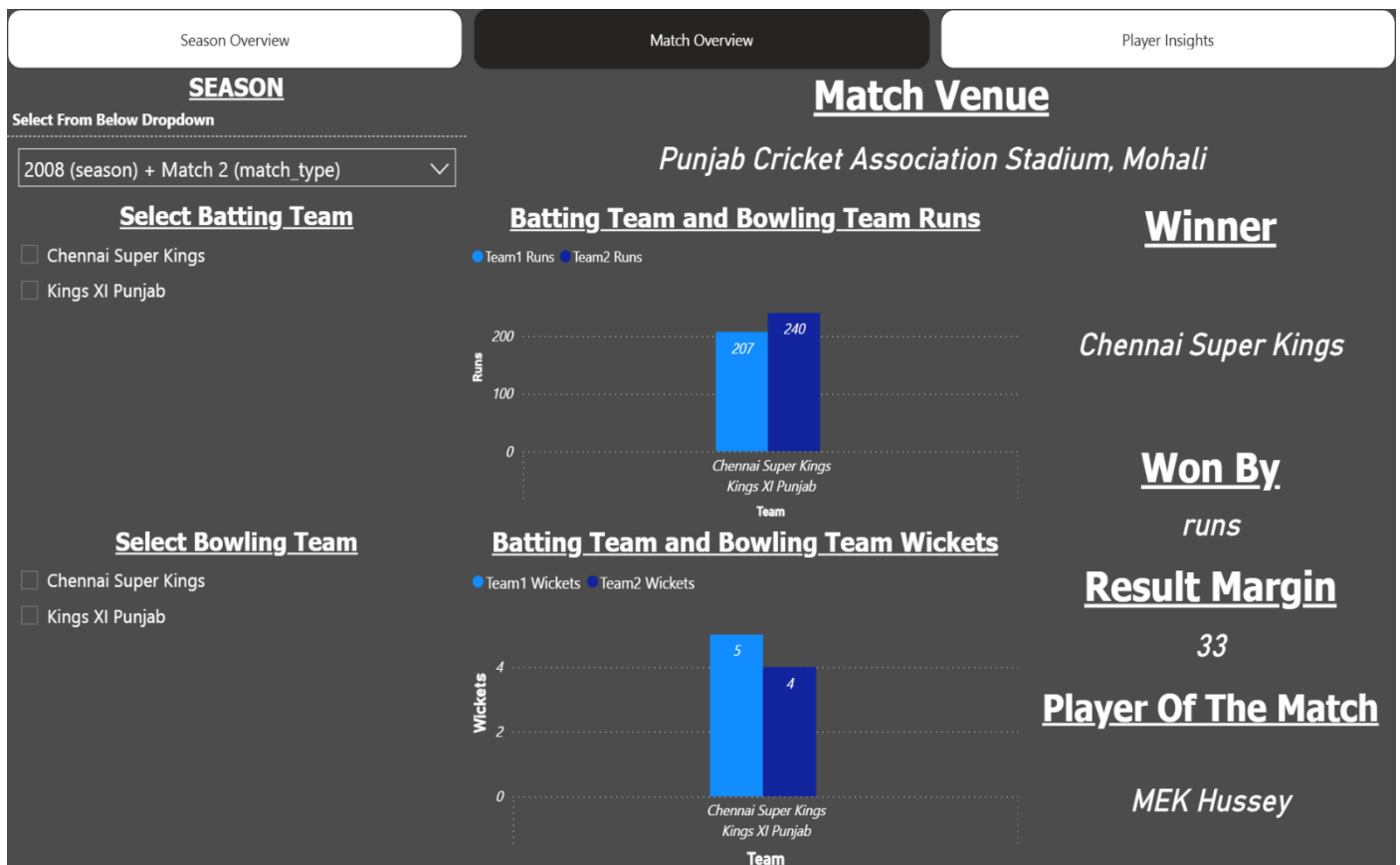


Fig 11.13. Power BI Dashboard Match Overview Page

Season Overview		Match Overview		Player Insights	
Batsmen Name		Total Runs in IPL Career		Total Wickets in IPL Career	
(Blank)		(Blank)		(Blank)	
Aakash Chopra		(Blank)		Aaron Finch	
Aaron Finch				Aavishkar Salvi	
AB de Villiers		(Blank)		Abdul Samad	
Abdul Basith				Abhimanyu thun	
Abdul Samad		(Blank)		Abhishek Jhunjhunwala	
Abdur Razzak				Abhishek Nayar	
Abhijeet Tomar		(Blank)		Abhishek Sharma	
Abhimanyu Mithun				Abu Nechim	
Abhinav Manohar		(Blank)		Adam Ine	
Abhinav Mukund				Adam Zampa	
Abhishek Jhunjhunwala		(Blank)		Adil Rashid	
Abhishek Nayar				Aditya Dole	
Abhishek Raut		(Blank)		Aiden Markram	
Abhishek Sharma				Ajantha Mendis	
Abishek Porel		(Blank)		Ajit Agarkar	
Abu Nechim				Ajit Chandila	
Adam Gilchrist		(Blank)		Akash Deep	
Adam Milne				Akash Madhwal	
Adam Voges		(Blank)		Akash Singh	
Adam Zampa				Akeal Hosein	

Fig 11.14. Power BI Dashboard Player Insight Page

11.2. Chapter Summary

The project delivers a strong, end-to-end data engineering pipeline:

- Data ingestion from HDFS to PostgreSQL is automated and monitored.
- Partitioning ensures efficient storage and deletion handling.
- Airflow orchestrates workflows and alerts stakeholders via email.
- Power BI dashboards enable interactive insights and data-driven decision-making.

These results show that the system is reliable, scalable, and capable of processing IPL data efficiently, providing both operational and analytical value.

Chapter 12

Discussion

This chapter discusses the significance of the project, the advantages of the chosen approach, and how it compares to typical ETL projects. The project integrates multiple tools—PySpark, Hadoop, Airflow, PostgreSQL, and Power BI—to build a strong, automated, and scalable IPL data pipeline.

12.1. Advantages of the Project

1. Automation and Efficiency

- Unlike manual or semi-automated ETL processes, this pipeline runs end-to-end without manual intervention.
- The IPL_db.py DAG checks for new files before execution, ensuring scripts run only when necessary, saving computation time and system resources.

2. Data Consistency and Integrity

- Partitioning in HDFS ensures efficient storage and retrieval.
- The monitoring DAG (partition_deletion.py) automatically deletes corresponding rows in PostgreSQL when partitions are removed, maintaining real-time consistency between distributed and relational storage.
- Most ETL projects do not handle deletion synchronization between source and destination, which can lead to stale or inconsistent data.

3. Scalability and Flexibility

- The pipeline can handle large IPL datasets across multiple seasons without major changes.
- Adding new tables, scripts, or even datasets from other domains requires minimal configuration.
- Using PySpark for data processing allows distributed computation, which is more efficient than single-node ETL tools.

4. Strong Orchestration with Airflow

- Airflow enables task dependency management, retries on failure, and email notifications for success or failure.
- Many ETL projects rely on cron jobs or scheduled scripts, which lack visibility, logging, and error handling.
- Airflow's DAG-based approach provides a clear structure for workflow execution and monitoring.

5. Enhanced Analytics and Visualization

- Processed data is stored in PostgreSQL and visualized using Power BI.
- Interactive dashboards with drill-down, filtering, and automatic refresh provide actionable insights.
- Many ETL projects stop at data processing without integrating visualization tools, limiting the usefulness of the processed data.

12.2. Why Using Multiple Tools Is Better Than a Single ETL Tool

1. Specialization

- Each tool in the stack is specialized:
 - **PySpark:** Distributed data processing
 - **Hadoop:** Scalable storage and partitioning
 - **Airflow:** Workflow orchestration and automation
 - **PostgreSQL:** Relational storage and structured querying
 - **Power BI:** Data visualization and insights
- Using a single ETL tool often compromises on performance, flexibility, or functionality in one or more areas.

2. Scalability and Performance

- Distributed computation and storage with PySpark and Hadoop allow handling large datasets efficiently.
- A single ETL tool may struggle with performance when processing massive datasets or real-time data streams.

3. Fault Tolerance and Monitoring

- Airflow provides retries, logging, and alerting, making the system strong against failures.
- Single ETL tools often have limited monitoring and may require custom solutions for alerts and logging.

4. Flexibility for Future Integration

- Multiple specialized tools make it easier to extend the pipeline to new data sources, cloud services, or visualization platforms.
- Using one tool often locks the project into specific technologies, limiting future scalability.

12.3. Comparison With Typical ETL Projects

Aspect	This Project	Typical ETL Project
Automation	Fully automated with file-checking logic	Often scheduled, may run unnecessary tasks
Data Consistency	Real-time partition monitoring and deletion sync	Rarely handles deletion sync between source and target
Scalability	Distributed processing with PySpark, scalable HDFS storage	Limited by single-node ETL tool
Orchestration	Airflow DAGs with retries, logging, and alerts	Cron jobs or built-in schedulers with limited monitoring
Visualization	Power BI dashboards with drill-down, filtering, auto-refresh	Often limited or absent
Flexibility	Easy to add tables, scripts, or datasets	Adding new sources often requires major redesign

Table 12.1. ETL Project comparison

12.4. Chapter Summary

The project demonstrates how integrating multiple specialized tools creates a more strong, scalable, and efficient ETL system compared to typical single-tool ETL projects. It ensures data consistency, automates workflow execution, monitors changes in near real-time, and provides actionable insights through visualization. This approach not only optimizes performance but also enhances flexibility for future expansion, making it superior to conventional ETL methods.

Chapter 13

Summary

This project presents the development of a comprehensive, automated data engineering pipeline for IPL data, integrating multiple tools—PySpark, Hadoop, Airflow, PostgreSQL, and Power BI. The primary objective was to build a system that efficiently processes large datasets, ensures data consistency, and provides actionable insights through visualization.

Key Highlights

1. End-to-End Automation

- PySpark scripts process IPL datasets (players, matches, deliveries) from HDFS to PostgreSQL.
- Airflow orchestrates the execution, running scripts only when new files are available and handling task dependencies automatically.

2. Data Consistency and Monitoring

- HDFS partitioning enables efficient storage and retrieval of data.
- The partition_deletion.py DAG continuously monitors partitions and ensures that deletions in HDFS are reflected in PostgreSQL, maintaining real-time consistency.

3. Scalability and Flexibility

- Distributed processing with PySpark and scalable storage with Hadoop allow the pipeline to handle growing datasets efficiently.
- The architecture is flexible, enabling the addition of new tables, scripts, or datasets with minimal changes.

4. Workflow Orchestration and Alerts

- Airflow provides strong orchestration, logging, retries, and email notifications for task success or failure.
- This ensures reliability and reduces manual intervention, unlike traditional ETL workflows.

5. Interactive Analytics with Power BI

- Processed data is visualized through interactive dashboards with filtering, drill-down, and automatic refresh.
- Dashboards provide insights into match statistics, player performance, and team analytics, bridging the gap between data engineering and business analysis.

6. Improved Approach Compared to Single-Tool ETL

- Using specialized tools for each task—data processing, storage, orchestration, database management, and visualization—enhances performance, monitoring, and scalability.

- The system demonstrates higher efficiency, flexibility, and actionable insight generation than typical single-tool ETL projects.

The project successfully implements a professional-grade, automated ETL pipeline that ensures efficient data processing, real-time monitoring, and actionable analytics. It highlights the advantages of using multiple specialized tools in a modern data engineering workflow, resulting in a reliable, scalable, and insightful system for IPL data analysis.

Chapter 14

Conclusion

This project successfully demonstrates the design and implementation of a complete, end-to-end data engineering pipeline for IPL data using PySpark, Hadoop, Airflow, PostgreSQL, and Power BI. The primary goal was to create an automated, scalable, and reliable ETL system that can process large datasets efficiently while maintaining data consistency and enabling actionable insights.

Key achievements of the project include:

1. **Automated ETL Pipeline:** The PySpark scripts, orchestrated via Airflow, process IPL data from HDFS to PostgreSQL automatically. The system intelligently checks for new files, running scripts only when required, saving time and resources.
2. **Data Consistency and Integrity:** Partitioning in HDFS and the monitoring DAG ensure that deletions are reflected in PostgreSQL, maintaining synchronization between distributed storage and relational tables.
3. **Scalable and Flexible Architecture:** The combination of PySpark for distributed computation, Hadoop for storage, and Airflow for orchestration allows the system to handle growing datasets efficiently and adapt easily to new tables or additional data sources.
4. **Real-Time Monitoring and Alerts:** The partition_deletion.py DAG provides near real-time monitoring of HDFS partitions, while Airflow's logging and email alerts ensure timely notification of any failures or successes in the pipeline.
5. **Actionable Insights with Power BI:** Processed data is visualized through interactive dashboards, enabling drill-down, filtering, and automatic updates, making it easy for stakeholders to analyze IPL statistics and trends.
6. **Improved Workflow Compared to Traditional ETL:** By leveraging multiple specialized tools instead of a single ETL tool, the project achieves better performance, flexibility, monitoring, and visualization capabilities.

Final Thoughts

The project illustrates the benefits of combining multiple modern data engineering tools to build a strong and efficient ETL system. It not only processes data efficiently but also ensures reliability, scalability, and real-time monitoring. Additionally, integrating visualization provides stakeholders with immediate access to insights, bridging the gap between data processing and business analysis. Overall, this project serves as a strong example of a practical, professional-grade ETL pipeline, demonstrating both technical proficiency and the ability to deliver actionable results.